

custom-os

A Bare-Metal Operating System for AArch64

Complete Technical Documentation

Language	Rust (nightly, <code>#![no_std]</code>)
Architecture	AArch64 (ARM64) / ARMv8-A
Kernel Type	Monolithic
Target Platform	QEMU virt machine (qemu-system-aarch64)
CPU Model	Cortex-A72
RAM	256 MB
Author	Abhishek Bhatia
Date	May 23, 2026
Lines of Code	~3,500+ (Rust + Assembly)

Table of Contents

1. Executive Summary
2. System Architecture
 - 2.1 Hardware Target & QEMU virt Memory Map
 - 2.2 Kernel Virtual Address Layout
 - 2.3 Project Structure
 - 2.4 Build Infrastructure
3. Phase 1: Boot Sequence & UART Console
 - 3.1 Assembly Boot Stub (boot.S)
 - 3.2 Linker Script
 - 3.3 PL011 UART Driver
 - 3.4 Console Macros
4. Phase 2: Exception Handling, GICv3 & Timer
 - 4.1 Exception Vector Table
 - 4.2 GICv3 Interrupt Controller
 - 4.3 ARM Generic Timer
5. Phase 3: Memory Management
 - 5.1 Physical Memory Manager (Bitmap Allocator)
 - 5.2 Virtual Memory Manager (4-Level Page Tables)
 - 5.3 MMU Initialization
6. Phase 4: Kernel Heap Allocator
7. Phase 5: Synchronization & Preemptive Scheduler
 - 7.1 SpinLock with Interrupt Masking
 - 7.2 Task Structure & Context
 - 7.3 Round-Robin Scheduler
 - 7.4 Assembly Context Switch
8. Phase 6: Device Drivers
 - 8.1 VirtIO MMIO Transport
 - 8.2 Split Virtqueue Implementation
 - 8.3 VirtIO Block Driver
 - 8.4 VirtIO Network Driver

9. Phase 7: Filesystem

9.1 Virtual File System (VFS)

9.2 FAT32 Read-Only Driver

9.3 RAM Filesystem

10. Phase 8: Network Stack

10.1 Ethernet Frame Layer

10.2 ARP

10.3 IPv4

10.4 ICMP (Ping)

10.5 UDP

10.6 TCP

11. Phase 9: System Call Interface

12. Phase 10: Userspace & Init Process

12.1 ELF Loader

12.2 User Page Table & ELO Transition

12.3 Init Process

13. Bugs, Issues & Fixes

14. Boot Output & Verification

15. Appendix: Complete Source Listing

1. Executive Summary

custom-os is a fully functional operating system kernel written from scratch in Rust, targeting the AArch64 (ARM64) architecture. The kernel runs on QEMU's *virt* machine with a Cortex-A72 CPU model and 256 MB of RAM. It implements all fundamental OS subsystems: boot initialization, interrupt handling, virtual memory with MMU, heap allocation, preemptive multitasking, device drivers, a filesystem layer, a TCP/IP network stack, a system call interface, and user-mode process execution.

The project demonstrates that a modern systems programming language like Rust can be used to build a complete operating system from bare metal, leveraging Rust's `#![no_std]` and `#![no_main]` attributes to operate without any runtime library. The kernel was developed in 10 incremental phases, each independently testable, progressing from serial output to a fully running userspace init process that communicates with the kernel via system calls.

Key Achievements

- Bare-metal AArch64 boot with assembly stub, BSS zeroing, and FP/SIMD enablement
- AArch64 exception vector table with 16 handler stubs using branch-to-stub pattern
- GICv3 interrupt controller initialization (Distributor + Redistributor + CPU Interface)
- ARM Generic Timer with 10ms tick driving preemptive scheduling
- 4-level page table MMU with identity + higher-half mapping (48-bit VA, 4KB granule)
- Bitmap-based physical page allocator managing 65,536 pages (256 MB)
- Linked-list kernel heap allocator with coalescing free blocks
- Preemptive round-robin scheduler with assembly `context_switch`
- VirtIO MMIO device drivers for block storage and networking (legacy v1 support)
- VFS abstraction with FAT32 read-only filesystem and in-memory RamFS
- Full TCP/IP network stack: Ethernet, ARP, IPv4, ICMP, UDP, TCP
- SVC-based system call interface dispatching from EL0 to EL1 handlers
- ELF loader with user page table creation and EL1-to-EL0 transition via `eret`
- Userspace init process printing 'Hello from userspace!' via `sys_write` syscall

2. System Architecture

The kernel follows a monolithic architecture where all subsystems (memory management, scheduling, drivers, filesystem, networking) run in a single address space at Exception Level 1 (EL1). Userspace processes run at EL0 with separate page tables (TTBR0) and communicate with the kernel through the SVC (Supervisor Call) instruction.

2.1 Hardware Target & QEMU virt Memory Map

The target platform is QEMU's virt machine with GICv3. The following QEMU command launches the OS:

QEMU Launch Command

```
qemu-system-aarch64 \
  -machine virt,gic-version=3 \
  -cpu cortex-a72 \
  -m 256M \
  -nographic -serial mon:stdio \
  -drive file=disk.img,format=raw,if=none,id=disk0 \
  -device virtio-blk-device,drive=disk0 \
  -netdev user,id=net0 \
  -device virtio-net-device,netdev=net0 \
  -kernel target/kernel.bin
```

Device / Region	Base Address	Size / IRQ	Description
GIC Distributor	0x0800_0000	--	Interrupt routing configuration
GIC Redistributor	0x080A_0000	--	Per-CPU interrupt config (GICv3)
PL011 UART	0x0900_0000	SPI 1 (INTID 33)	Serial console I/O
VirtIO MMIO x32	0x0A00_0000	SPI 16-47	32 slots, stride 0x200
PCIe ECAM	0x3F00_0000	--	PCI configuration space
RAM	0x4000_0000	256 MB	Main system memory

2.2 Kernel Virtual Address Layout

The MMU is configured with separate page tables for the lower half (TTBR0, identity-mapped during boot, later used for userspace) and upper half (TTBR1, higher-half kernel). Both use 48-bit virtual addresses with 4KB granule.

Region	Virtual Address	Mapping
MMIO (identity)	0x0000_0000 - 0x3FFF_FFFF	Device memory (nGnRnE)

Region	Virtual Address	Mapping
RAM (identity)	0x4000_0000 - 0x4FFF_FFFF	Normal memory (WB-WA)
MMIO (higher-half)	0xFFFF_0000_0000_0000+	Mirror of MMIO region
Kernel code+data	0xFFFF_0000_4008_0000+	Mirror of kernel in RAM
Userspace code	0x0040_0000+	User ELF segments (TTBR0)
Userspace stack	0x7FFF_C000 - 0x7FFF_FFFF	16 KB user stack (TTBR0)

2.3 Project Structure

Project Directory Layout

```

custom-os/
  Cargo.toml           # Workspace root
  Makefile             # Build + run targets
  rust-toolchain.toml  # Nightly + aarch64-unknown-none
  .cargo/config.toml   # Cross-compilation config
  disk.img             # 64MB FAT32 disk image
  kernel/
    Cargo.toml
    linker.ld          # Kernel linker script
    init.elf           # Embedded userspace binary
    src/
      main.rs          # Kernel entry point
      console.rs       # print!/println! macros
      arch/aarch64/
        mod.rs, boot.S, exceptions.rs, gic.rs, timer.rs
      mm/
        mod.rs, pmm.rs, vmm.rs, heap.rs
      sync/
        mod.rs, spinlock.rs
      sched/
        mod.rs, task.rs, scheduler.rs, process.rs
      drivers/
        mod.rs, uart.rs, pci.rs
      virtio/ (mod.rs, queue.rs, block.rs, net.rs)
      fs/
        mod.rs, vfs.rs, fat32.rs, ramfs.rs
      net/
        mod.rs, ethernet.rs, arp.rs, ipv4.rs,
        icmp.rs, udp.rs, tcp.rs
      syscall/
        mod.rs, handlers.rs
    userspace/init/
      Cargo.toml, linker.ld, src/main.rs
    tools/
      mkimage.sh       # FAT32 disk image creator

```

2.4 Build Infrastructure

The project uses Rust's nightly toolchain targeting `aarch64-unknown-none` (bare-metal AArch64). The workspace is configured with `panic = "abort"` and LTO enabled for minimal binary size. The kernel binary is extracted from the ELF using `rust-objcopy`.

rust-toolchain.toml

```
[toolchain]
channel = "nightly"
targets = ["aarch64-unknown-none"]
components = ["rust-src", "llvm-tools-preview", "rustfmt", "clippy"]
```

.cargo/config.toml

```
[build]
target = "aarch64-unknown-none"

[target.aarch64-unknown-none]
rustflags = ["-C", "link-arg=-Tkernel/linker.ld"]
```

Makefile

```
KERNEL_ELF = target/aarch64-unknown-none/release/kernel
KERNEL_BIN = target/kernel.bin
DISK_IMG = disk.img

QEMU = qemu-system-aarch64
QEMU_ARGS = -machine virt,gic-version=3 -cpu cortex-a72 -m 256M -nographic -serial mon:stdio
QEMU_DEVICES = -drive file=$(DISK_IMG),format=raw,if=none,id=disk0 \
■-device virtio-blk-device,drive=disk0 \
■-netdev user,id=net0 \
■-device virtio-net-device,netdev=net0

.PHONY: build run run-bare debug clean clippy disk

build:
■cargo build --release

$(KERNEL_BIN): build
■rust-objcopy --strip-all -O binary $(KERNEL_ELF) $(KERNEL_BIN)

disk:
■./tools/mkimage.sh $(DISK_IMG)

run: $(KERNEL_BIN) $(DISK_IMG)
■$(QEMU) $(QEMU_ARGS) $(QEMU_DEVICES) -kernel $(KERNEL_BIN)

run-bare: $(KERNEL_BIN)
■$(QEMU) $(QEMU_ARGS) -kernel $(KERNEL_BIN)

debug: $(KERNEL_BIN) $(DISK_IMG)
■$(QEMU) $(QEMU_ARGS) $(QEMU_DEVICES) -kernel $(KERNEL_BIN) -S -s

clean:
■cargo clean
■rm -f $(KERNEL_BIN)

clippy:
■cargo clippy --release
```

3. Phase 1: Boot Sequence & UART Console

The boot sequence begins when QEMU loads the raw kernel binary at physical address 0x4008_0000 and jumps to `_start`. The assembly boot stub performs critical early initialization before handing control to Rust.

3.1 Assembly Boot Stub (boot.S)

The boot stub performs five essential tasks: (1) save the DTB pointer from `x0`, (2) check CPU ID and park secondary cores, (3) zero the BSS section, (4) enable FP/SIMD via `CPACR_EL1`, and (5) set the stack pointer and call `kernel_main`.

kernel/src/arch/aarch64/boot.S

```
.section .text.boot
.global _start

_start:
    // x0 contains DTB pointer from QEMU, save it
    mov     x19, x0

    // Only let CPU 0 through, park all others
    mrs     x1, mpidr_el1
    and     x1, x1, #0xFF
    cbnz    x1, .Lpark

    // Zero BSS
    adrp    x1, __bss_start
    add     x1, x1, :lo12:__bss_start
    adrp    x2, __bss_end
    add     x2, x2, :lo12:__bss_end
.Lbss_loop:
    cmp     x1, x2
    b.ge    .Lbss_done
    stp     xzr, xzr, [x1], #16
    b       .Lbss_loop
.Lbss_done:

    // Enable FP/SIMD (CPACR_EL1.FPEN = 0b11)
    mov     x1, #(3 << 20)
    msr     CPACR_EL1, x1
    isb

    // Set up the stack
    adrp    x1, __stack_top
    add     x1, x1, :lo12:__stack_top
    mov     sp, x1

    // Jump to Rust entry point with DTB pointer as argument
    mov     x0, x19
    bl      kernel_main

    // Should never return, but just in case
.Lpark:
    wfe
    b       .Lpark
```

3.2 Linker Script

The linker script places `.text.boot` at the QEMU load address `0x4008_0000`, followed by code, rodata, data, and BSS sections. It exports symbols for BSS zeroing and defines a 64KB boot stack.

kernel/linker.ld

```
ENTRY(_start)

SECTIONS
{
    . = 0x40080000;

    .text.boot : {
        KEEP(*(.text.boot))
    }

    .text : {
        *(.text .text.*)
    }

    . = ALIGN(4096);
    .rodata : {
        *(.rodata .rodata.*)
    }

    . = ALIGN(4096);
    .data : {
        *(.data .data.*)
    }

    . = ALIGN(4096);
    __bss_start = .;
    .bss : {
        *(.bss .bss.*)
        *(COMMON)
    }
    __bss_end = .;

    . = ALIGN(4096);
    __kernel_end = .;

    . = . + 0x10000;
    . = ALIGN(16);
    __stack_top = .;
}
```

3.3 PL011 UART Driver

The PL011 UART at `0x0900_0000` provides the serial console. The driver implements `core::fmt::Write`, checking the TXFF (transmit FIFO full) flag before writing each character.

kernel/src/drivers/uart.rs

```
use core::fmt;

const UART0_BASE: usize = 0x0900_0000;

const UART_DR: usize = 0x00;
const UART_FR: usize = 0x18;
const UART_FR_TXFF: u32 = 1 <&lt; 5;
```

```

pub struct Pl011 {
    base: usize,
}

impl Pl011 {
    pub const fn new(base: usize) -> Self {
        Self { base }
    }

    fn wait_tx_ready(&self) {
        unsafe {
            while (core::ptr::read_volatile((self.base + UART_FR) as *const u32) & UART_FR_TXFF)
                != 0
            {
                core::hint::spin_loop();
            }
        }
    }

    pub fn putc(&self, c: u8) {
        self.wait_tx_ready();
        unsafe {
            core::ptr::write_volatile((self.base + UART_DR) as *mut u8, c);
        }
    }
}

impl fmt::Write for Pl011 {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        for byte in s.bytes() {
            if byte == b'\n' {
                self.putc(b'\r');
            }
            self.putc(byte);
        }
        Ok(())
    }
}

pub fn uart() -> Pl011 {
    Pl011::new(UART0_BASE)
}

```

3.4 Console Macros

kernel/src/console.rs

```

use core::fmt;
use crate::drivers::uart;

pub fn _print(args: fmt::Arguments) {
    use fmt::Write;
    uart::uart().write_fmt(args).unwrap();
}

#[macro_export]
macro_rules! print {
    ($($arg:tt)*) => { ($crate::console::_print(format_args!($($arg)*))); }
}

#[macro_export]
macro_rules! println {
    () => { ($crate::print!("\n")); }
    ($($arg:tt)*) => { ($crate::print!("{}", format_args!($($arg)*))); }
}

```

}

4. Phase 2: Exception Handling, GICv3 & Timer

4.1 Exception Vector Table

AArch64 requires a 2KB-aligned vector table with 16 entries, each 128 bytes. Since the full handler code exceeds 128 bytes, each entry contains a single b (branch) instruction to a handler stub placed outside the table. Each stub uses SAVE_CONTEXT/RESTORE_CONTEXT macros that save all 31 general-purpose registers plus ELR_EL1, SPSR_EL1, ESR_EL1, and FAR_EL1 into an ExceptionContext struct on the stack.

The four exception classes at each of four source levels give 16 vectors total: Synchronous, IRQ, FIQ, and SError for Current EL with SP_EL0, Current EL with SP_ELx, Lower EL AArch64, and Lower EL AArch32.

kernel/src/arch/aarch64/exceptions.rs

```
use core::arch::global_asm;

#[repr(C)]
pub struct ExceptionContext {
    pub gpr: [u64; 31],
    pub elr_el1: u64,
    pub spsr_el1: u64,
    pub esr_el1: u64,
    pub far_el1: u64,
}

const CONTEXT_SIZE: usize = core::mem::size_of::<ExceptionContext>();

global_asm!(
    "
    .macro SAVE_CONTEXT
        sub    sp, sp, #{size}
        stp    x0, x1, [sp, #16 * 0]
        stp    x2, x3, [sp, #16 * 1]
        stp    x4, x5, [sp, #16 * 2]
        stp    x6, x7, [sp, #16 * 3]
        stp    x8, x9, [sp, #16 * 4]
        stp    x10, x11, [sp, #16 * 5]
        stp    x12, x13, [sp, #16 * 6]
        stp    x14, x15, [sp, #16 * 7]
        stp    x16, x17, [sp, #16 * 8]
        stp    x18, x19, [sp, #16 * 9]
        stp    x20, x21, [sp, #16 * 10]
        stp    x22, x23, [sp, #16 * 11]
        stp    x24, x25, [sp, #16 * 12]
        stp    x26, x27, [sp, #16 * 13]
        stp    x28, x29, [sp, #16 * 14]
        str    x30, [sp, #16 * 15]

        mrs    x0, elr_el1
        mrs    x1, spsr_el1
        mrs    x2, esr_el1
        mrs    x3, far_el1
        stp    x0, x1, [sp, #16 * 15 + 8]
        stp    x2, x3, [sp, #16 * 16 + 8]
    .endm
```

```

.macro RESTORE_CONTEXT
    ldp    x0, x1, [sp, #16 * 15 + 8]
    msr    elr_el1, x0
    msr    spsr_el1, x1

    ldp    x0, x1, [sp, #16 * 0]
    ldp    x2, x3, [sp, #16 * 1]
    ldp    x4, x5, [sp, #16 * 2]
    ldp    x6, x7, [sp, #16 * 3]
    ldp    x8, x9, [sp, #16 * 4]
    ldp    x10, x11, [sp, #16 * 5]
    ldp    x12, x13, [sp, #16 * 6]
    ldp    x14, x15, [sp, #16 * 7]
    ldp    x16, x17, [sp, #16 * 8]
    ldp    x18, x19, [sp, #16 * 9]
    ldp    x20, x21, [sp, #16 * 10]
    ldp    x22, x23, [sp, #16 * 11]
    ldp    x24, x25, [sp, #16 * 12]
    ldp    x26, x27, [sp, #16 * 13]
    ldp    x28, x29, [sp, #16 * 14]
    ldr    x30, [sp, #16 * 15]
    add    sp, sp, #{size}
.endm

.macro HANDLER_STUB handler
    SAVE_CONTEXT
    mov    x0, sp
    bl     \\handler
    RESTORE_CONTEXT
    eret
.endm

// Vector table: each entry is only 0x80 (128) bytes, so just branch out
.balign 0x800
.global __exception_vectors
__exception_vectors:

// Current EL with SP_EL0
.balign 0x80
    b     __exc_sync_cur_sp0
.balign 0x80
    b     __exc_irq_cur_sp0
.balign 0x80
    b     __exc_fiq_cur_sp0
.balign 0x80
    b     __exc_serror_cur_sp0

// Current EL with SP_ELx
.balign 0x80
    b     __exc_sync_cur_spx
.balign 0x80
    b     __exc_irq_cur_spx
.balign 0x80
    b     __exc_fiq_cur_spx
.balign 0x80
    b     __exc_serror_cur_spx

// Lower EL, AArch64
.balign 0x80
    b     __exc_sync_lower64
.balign 0x80
    b     __exc_irq_lower64
.balign 0x80
    b     __exc_fiq_lower64
.balign 0x80

```

```

        b        __exc_serror_lower64

// Lower EL, AArch32
.balign 0x80
    b        __exc_sync_lower32
.balign 0x80
    b        __exc_irq_lower32
.balign 0x80
    b        __exc_fiq_lower32
.balign 0x80
    b        __exc_serror_lower32

// Handler stubs (outside the vector table, no size limit)
__exc_sync_cur_sp0:
    HANDLER_STUB exception_sync_current_el_sp0
__exc_irq_cur_sp0:
    HANDLER_STUB exception_irq_current_el_sp0
__exc_fiq_cur_sp0:
    HANDLER_STUB exception_fiq_current_el_sp0
__exc_serror_cur_sp0:
    HANDLER_STUB exception_serror_current_el_sp0

__exc_sync_cur_spx:
    HANDLER_STUB exception_sync_current_el_spx
__exc_irq_cur_spx:
    HANDLER_STUB exception_irq_current_el_spx
__exc_fiq_cur_spx:
    HANDLER_STUB exception_fiq_current_el_spx
__exc_serror_cur_spx:
    HANDLER_STUB exception_serror_current_el_spx

__exc_sync_lower64:
    HANDLER_STUB exception_sync_lower_el_aarch64
__exc_irq_lower64:
    HANDLER_STUB exception_irq_lower_el_aarch64
__exc_fiq_lower64:
    HANDLER_STUB exception_fiq_lower_el_aarch64
__exc_serror_lower64:
    HANDLER_STUB exception_serror_lower_el_aarch64

__exc_sync_lower32:
    HANDLER_STUB exception_sync_lower_el_aarch32
__exc_irq_lower32:
    HANDLER_STUB exception_irq_lower_el_aarch32
__exc_fiq_lower32:
    HANDLER_STUB exception_fiq_lower_el_aarch32
__exc_serror_lower32:
    HANDLER_STUB exception_serror_lower_el_aarch32
",
    size = const CONTEXT_SIZE,
);

fn default_handler(ctx: &ExceptionContext, kind: &str) {
    crate::println!(
        "Unhandled exception: {}",
        kind
    );
    crate::println!(
        "  ELR_EL1:  {:#018x}  SPSR_EL1: {:#018x}",
        ctx.elr_el1,
        ctx.spsr_el1
    );
    crate::println!(
        "  ESR_EL1:  {:#018x}  FAR_EL1:  {:#018x}",
        ctx.esr_el1,

```

```

        ctx.far_el1
    );
    loop {
        core::hint::spin_loop();
    }
}

#[no_mangle]
extern "C" fn exception_sync_current_el_sp0(ctx: &ExceptionContext) {
    default_handler(ctx, "Synchronous (Current EL, SP_EL0)");
}

#[no_mangle]
extern "C" fn exception_irq_current_el_sp0(ctx: &ExceptionContext) {
    default_handler(ctx, "IRQ (Current EL, SP_EL0)");
}

#[no_mangle]
extern "C" fn exception_fiq_current_el_sp0(ctx: &ExceptionContext) {
    default_handler(ctx, "FIQ (Current EL, SP_EL0)");
}

#[no_mangle]
extern "C" fn exception_serror_current_el_sp0(ctx: &ExceptionContext) {
    default_handler(ctx, "SError (Current EL, SP_EL0)");
}

#[no_mangle]
extern "C" fn exception_sync_current_el_spx(ctx: &mut ExceptionContext) {
    let ec = (ctx.esr_el1 >>> 26) & 0x3F;
    match ec {
        0x15 => {
            // SVC from EL1 (kernel-mode syscall test)
            crate::syscall::dispatch(ctx);
        }
        _ => {
            default_handler(ctx, "Synchronous (Current EL, SP_ELx)");
        }
    }
}

#[no_mangle]
extern "C" fn exception_irq_current_el_spx(_ctx: &ExceptionContext) {
    super::gic::handle_irq();
}

#[no_mangle]
extern "C" fn exception_fiq_current_el_spx(ctx: &ExceptionContext) {
    default_handler(ctx, "FIQ (Current EL, SP_ELx)");
}

#[no_mangle]
extern "C" fn exception_serror_current_el_spx(ctx: &ExceptionContext) {
    default_handler(ctx, "SError (Current EL, SP_ELx)");
}

#[no_mangle]
extern "C" fn exception_sync_lower_el_aarch64(ctx: &mut ExceptionContext) {
    let ec = (ctx.esr_el1 >>> 26) & 0x3F;
    match ec {
        0x15 => {
            crate::syscall::dispatch(ctx);
        }
        _ => {
            crate::println!(

```

```

        "[exc] Lower EL sync: EC={:#x} ESR={:#x} ELR={:#x} FAR={:#x}",
        ec, ctx.esr_el1, ctx.elr_el1, ctx.far_el1
    );
    default_handler(ctx, "Synchronous (Lower EL, AArch64)");
}
}
}

#[no_mangle]
extern "C" fn exception_irq_lower_el_aarch64(_ctx: &mut ExceptionContext) {
    super::gic::handle_irq();
}

#[no_mangle]
extern "C" fn exception_fiq_lower_el_aarch64(ctx: &ExceptionContext) {
    default_handler(ctx, "FIQ (Lower EL, AArch64)");
}

#[no_mangle]
extern "C" fn exception_serror_lower_el_aarch64(ctx: &ExceptionContext) {
    default_handler(ctx, "SError (Lower EL, AArch64)");
}

#[no_mangle]
extern "C" fn exception_sync_lower_el_aarch32(ctx: &ExceptionContext) {
    default_handler(ctx, "Synchronous (Lower EL, AArch32)");
}

#[no_mangle]
extern "C" fn exception_irq_lower_el_aarch32(ctx: &ExceptionContext) {
    default_handler(ctx, "IRQ (Lower EL, AArch32)");
}

#[no_mangle]
extern "C" fn exception_fiq_lower_el_aarch32(ctx: &ExceptionContext) {
    default_handler(ctx, "FIQ (Lower EL, AArch32)");
}

#[no_mangle]
extern "C" fn exception_serror_lower_el_aarch32(ctx: &ExceptionContext) {
    default_handler(ctx, "SError (Lower EL, AArch32)");
}

pub fn init() {
    unsafe {
        core::arch::asm!(
            "adrp x0, __exception_vectors",
            "add x0, x0, :lo12:__exception_vectors",
            "msr vbar_el1, x0",
            "isb",
            out("x0") _,
        );
    }
    crate::println!("[exceptions] Vector table installed");
}

```

4.2 GICv3 Interrupt Controller

The Generic Interrupt Controller version 3 (GICv3) manages all hardware interrupts. Initialization requires three components: the Distributor (GICD) for global SPI routing, the Redistributor (GICR) for per-CPU PPI/SGI configuration, and the CPU Interface via ICC system registers. Key design decision: EOI (End of

Interrupt) is written **before** dispatching to the handler, because the timer handler may call `schedule()` which performs a context switch and never returns to the EOI write.

kernel/src/arch/aarch64/gic.rs

```
use core::ptr::{read_volatile, write_volatile};

const GICD_BASE: usize = 0x0800_0000;
const GICR_BASE: usize = 0x080A_0000;

// Distributor registers
const GICD_CTLR: usize = GICD_BASE;
const GICD_TYPER: usize = GICD_BASE + 0x0004;
const GICD_IGROUPR: usize = GICD_BASE + 0x0080;
const GICD_ISENABLER: usize = GICD_BASE + 0x0100;
const GICD_ICENABLER: usize = GICD_BASE + 0x0180;
const GICD_IPRIORITYR: usize = GICD_BASE + 0x0400;
const GICD_ICFGR: usize = GICD_BASE + 0x0C00;

// Redistributor registers (SGI frame at +0x10000)
const GICR_WAKER: usize = GICR_BASE + 0x0014;
const GICR_SGI_BASE: usize = GICR_BASE + 0x10000;
const GICR_IGROUPR0: usize = GICR_SGI_BASE + 0x0080;
const GICR_ISENABLER0: usize = GICR_SGI_BASE + 0x0100;
const GICR_IPRIORITYR: usize = GICR_SGI_BASE + 0x0400;

// Timer PPI
const TIMER_IRQ: u32 = 27;

unsafe fn reg_write(addr: usize, val: u32) {
    write_volatile(addr as *mut u32, val);
}

unsafe fn reg_read(addr: usize) -> u32 {
    read_volatile(addr as *const u32)
}

fn gicd_init() {
    unsafe {
        // Disable distributor
        reg_write(GICD_CTLR, 0);

        // Read number of interrupt lines
        let typer = reg_read(GICD_TYPER);
        let num_irqs = ((typer & 0x1F) + 1) * 32;

        // Configure all SPIs: Group 1, priority 0xA0, disabled
        let num_regs = (num_irqs / 32) as usize;
        for i in 1..num_regs {
            reg_write(GICD_IGROUPR + i * 4, 0xFFFF_FFFF);
            reg_write(GICD_ICENABLER + i * 4, 0xFFFF_FFFF);
        }

        // Set all SPI priorities to 0xA0
        for i in 8..(num_irqs as usize / 4) {
            reg_write(GICD_IPRIORITYR + i * 4, 0xA0A0_A0A0);
        }

        // Enable distributor with affinity routing (ARE_NS) and Group 1 NS
        reg_write(GICD_CTLR, (1 << 4) | (1 << 1));
    }
}

fn gicr_init() {
    unsafe {
```

```

    // Wake up redistributor
    let waker = reg_read(GICR_WAKER);
    reg_write(GICR_WAKER, waker & !(1 <<< 1)); // Clear ProcessorSleep

    // Wait for ChildrenAsleep to clear
    while (reg_read(GICR_WAKER) & (1 <<< 2)) != 0 {
        core::hint::spin_loop();
    }

    // Set SGIs/PPIs to Group 1
    reg_write(GICR_IGROUPR0, 0xFFFF_FFFF);

    // Set PPI priorities to 0xA0
    for i in 0..8 {
        reg_write(GICR_IPRIORITYR + i * 4, 0xA0A0_A0A0);
    }

    // Enable timer PPI (INTID 27)
    reg_write(GICR_ISENBLER0, 1 <<< TIMER_IRQ);
}

fn cpu_interface_init() {
    unsafe {
        // Enable system register interface
        core::arch::asm!(
            "mrs x0, ICC_SRE_EL1",
            "orr x0, x0, #0x7",
            "msr ICC_SRE_EL1, x0",
            "isb",
            out("x0") _,
        );

        // Set priority mask to allow all priorities
        core::arch::asm!("msr ICC_PMR_EL1, {}", in(reg) 0xFFu64);

        // EOI mode: drop priority and deactivate
        core::arch::asm!("msr ICC_CTLR_EL1, {}", in(reg) 0u64);

        // Enable Group 1 interrupts
        core::arch::asm!("msr ICC_IGRPEN1_EL1, {}", in(reg) 1u64);

        // Unmask IRQs (clear I bit in DAIF)
        core::arch::asm!("msr DAIFClr, #0b0010");
    }
}

pub fn init() {
    gicd_init();
    gicr_init();
    cpu_interface_init();
    crate::println!("[gic] GICv3 initialized");
}

pub fn handle_irq() {
    let intid: u64;
    unsafe {
        core::arch::asm!("mrs {}, ICC_IAR1_EL1", out(reg) intid);
    }

    // Write EOI early so the GIC can accept new interrupts even if
    // the handler triggers a context switch and never returns here
    unsafe {
        core::arch::asm!("msr ICC_EOIR1_EL1, {}", in(reg) intid);
    }
}

```

```

match intid as u32 {
    TIMER_IRQ => {
        super::timer::handle_tick();
    }
    1023 => { }
    id => {
        crate::println!("[gic] Unhandled IRQ: {}", id);
    }
}
}

```

4.3 ARM Generic Timer

The virtual EL1 timer (INTID 27, PPI) generates 10ms ticks. The timer frequency is read from CNTFRQ_EL0, and CNTV_TVAL_EL0 is set to freq/100 for each tick. The tick handler re-arms the timer and calls the scheduler for preemptive multitasking.

kernel/src/arch/aarch64/timer.rs

```

use core::sync::atomic::{AtomicU64, Ordering};

static TICKS: AtomicU64 = AtomicU64::new(0);

const TICKS_PER_SECOND: u64 = 100; // 10ms per tick

fn counter_frequency() -> u64 {
    let freq: u64;
    unsafe {
        core::arch::asm!("mrs {}, CNTFRQ_EL0", out(reg) freq);
    }
    freq
}

pub fn init() {
    let freq = counter_frequency();
    let tval = freq / TICKS_PER_SECOND;

    unsafe {
        // Set timer value
        core::arch::asm!("msr CNTV_TVAL_EL0, {}", in(reg) tval);
        // Enable timer: ENABLE=1, IMASK=0
        core::arch::asm!("msr CNTV_CTL_EL0, {}", in(reg) 1u64);
    }

    crate::println!("[timer] Initialized at {} Hz, tick every {}ms", freq, 1000 / TICKS_PER_SECOND);
}

pub fn handle_tick() {
    let tick = TICKS.fetch_add(1, Ordering::Relaxed) + 1;

    // Re-arm the timer
    let freq = counter_frequency();
    let tval = freq / TICKS_PER_SECOND;
    unsafe {
        core::arch::asm!("msr CNTV_TVAL_EL0, {}", in(reg) tval);
    }

    if tick % (TICKS_PER_SECOND * 1) == 0 {
        crate::println!("[timer] {} seconds elapsed", tick / TICKS_PER_SECOND);
    }

    // Trigger scheduler
    crate::sched::schedule();
}

```

```
}  
pub fn ticks() -&gt; u64 {  
    TICKS.load(Ordering::Relaxed)  
}
```

5. Phase 3: Memory Management

5.1 Physical Memory Manager (Bitmap Allocator)

The PMM uses a bitmap where each bit represents one 4KB page. For 256MB of RAM starting at 0x4000_0000, there are 65,536 pages requiring a 1024-entry u64 bitmap (8KB). `alloc_page()` scans for the first free bit using `trailing_zeros()`, zeros the allocated page, and marks it used. Pages occupied by the kernel image are marked during initialization.

kernel/src/mm/pmm.rs

```
use super::{PhysAddr, PAGE_SIZE, PAGE_SHIFT};
use core::sync::atomic::{AtomicUsize, Ordering};

const MAX_PAGES: usize = 65536; // 256MB / 4KB
const BITMAP_SIZE: usize = MAX_PAGES / 64;

static mut BITMAP: [u64; BITMAP_SIZE] = [0; BITMAP_SIZE];
static mut BASE_ADDR: usize = 0;
static mut NUM_PAGES: usize = 0;
static FREE_COUNT: AtomicUsize = AtomicUsize::new(0);

extern "C" {
    static __kernel_end: u8;
}

pub unsafe fn init(ram_base: usize, ram_size: usize) {
    BASE_ADDR = ram_base;
    NUM_PAGES = ram_size / PAGE_SIZE;

    // Mark all pages as free (0 = free, 1 = used)
    for i in 0..BITMAP_SIZE {
        BITMAP[i] = 0;
    }

    // Mark pages before and including the kernel as used
    let kernel_end = &__kernel_end as *const u8 as usize;
    // Also account for the boot stack (64KB after __kernel_end in linker script)
    let used_end = kernel_end + 0x10000;
    let used_pages = (used_end - ram_base + PAGE_SIZE - 1) / PAGE_SIZE;

    for i in 0..used_pages {
        mark_used(i);
    }

    FREE_COUNT.store(NUM_PAGES - used_pages, Ordering::Relaxed);
}

unsafe fn mark_used(page_idx: usize) {
    let word = page_idx / 64;
    let bit = page_idx % 64;
    BITMAP[word] |= 1 << bit;
}

unsafe fn mark_free(page_idx: usize) {
    let word = page_idx / 64;
    let bit = page_idx % 64;
    BITMAP[word] &= !(1 << bit);
}
```

```

}

fn is_free(page_idx: usize) -> bool {
    unsafe {
        let word = page_idx / 64;
        let bit = page_idx % 64;
        (BITMAP[word] & (1 << bit)) == 0
    }
}

pub fn alloc_page() -> Option<PhysAddr> {
    unsafe {
        for word_idx in 0..BITMAP_SIZE {
            if BITMAP[word_idx] != u64::MAX {
                let bit = (!BITMAP[word_idx]).trailing_zeros() as usize;
                let page_idx = word_idx * 64 + bit;
                if page_idx >= NUM_PAGES {
                    return None;
                }
                mark_used(page_idx);
                FREE_COUNT.fetch_sub(1, Ordering::Relaxed);
                let addr = BASE_ADDR + page_idx * PAGE_SIZE;

                // Zero the page
                core::ptr::write_bytes(addr as *mut u8, 0, PAGE_SIZE);

                return Some(PhysAddr(addr));
            }
        }
    }
    None
}

pub fn free_page(addr: PhysAddr) {
    unsafe {
        let page_idx = (addr.0 - BASE_ADDR) >> PAGE_SHIFT;
        if page_idx < NUM_PAGES && !is_free(page_idx) {
            mark_free(page_idx);
            FREE_COUNT.fetch_add(1, Ordering::Relaxed);
        }
    }
}

pub fn free_pages() -> usize {
    FREE_COUNT.load(Ordering::Relaxed)
}

pub fn total_pages() -> usize {
    unsafe { NUM_PAGES }
}

```

5.2 Virtual Memory Manager (4-Level Page Tables)

AArch64 uses 4-level page tables (L0-L3) with 512 entries per table and 4KB granule. Virtual address bits are extracted as: L0 = VA[47:39], L1 = VA[38:30], L2 = VA[29:21], L3 = VA[20:12]. The boot page tables use 2MB block mappings at L2 for simplicity. The `map_page()` function creates full 4-level mappings with L3 page descriptors, used for userspace process page tables.

Page table entry flags include: Valid (bit 0), Table/Block (bit 1), AttrIndx (bits 4:2) for MAIR index selection, AP (bits 7:6) for access permissions, SH (bits 9:8) for shareability, AF (bit 10) for access flag, and

UXN/PXN (bits 54:53) for execute-never.

kernel/src/mm/vmm.rs

```
use super::{PhysAddr, VirtAddr, KERNEL_VA_BASE};
use super::pmm;

const ENTRIES_PER_TABLE: usize = 512;

#[repr(C, align(4096))]
pub struct PageTable {
    pub entries: [u64; ENTRIES_PER_TABLE],
}

// Page table entry flags
const PTE_VALID: u64 = 1 <<< 0;
const PTE_TABLE: u64 = 1 <<< 1; // L0-L2: table descriptor; L3: page descriptor
const PTE_AF: u64 = 1 <<< 10; // Access flag
const PTE_SH_INNER: u64 = 3 <<< 8; // Inner shareable
const PTE_AP_RW_EL1: u64 = 0 <<< 6;
const PTE_AP_RO_EL1: u64 = 2 <<< 6;
const PTE_AP_RW_ALL: u64 = 1 <<< 6;
const PTE_UXN: u64 = 1 <<< 54;
const PTE_PXN: u64 = 1 <<< 53;

// MAIR attribute indices
const MAIR_NORMAL_IDX: u64 = 0;
const MAIR_DEVICE_IDX: u64 = 1;

fn attr_index(idx: u64) -> u64 {
    idx <<< 2
}

pub struct MappingFlags {
    pub writable: bool,
    pub executable: bool,
    pub user: bool,
    pub device: bool,
}

impl MappingFlags {
    pub fn kernel_code() -> Self {
        Self { writable: false, executable: true, user: false, device: false }
    }

    pub fn kernel_data() -> Self {
        Self { writable: true, executable: false, user: false, device: false }
    }

    pub fn kernel_device() -> Self {
        Self { writable: true, executable: false, user: false, device: true }
    }
}

fn flags_to_pte(flags: &MappingFlags) -> u64 {
    let mut pte = PTE_VALID | PTE_TABLE | PTE_AF;

    if flags.device {
        pte |= attr_index(MAIR_DEVICE_IDX);
    } else {
        pte |= attr_index(MAIR_NORMAL_IDX) | PTE_SH_INNER;
    }

    if flags.user {
        if flags.writable {
```

```

        pte |= PTE_AP_RW_ALL; // AP[2:1]=01: EL0+EL1 read-write
    } else {
        pte |= PTE_AP_RW_ALL | PTE_AP_R0_EL1; // AP[2:1]=11: EL0+EL1 read-only
    }
    if !flags.executable {
        pte |= PTE_UXN;
    }
    pte |= PTE_PXN; // never execute user pages at EL1
} else {
    if flags.writable {
        pte |= PTE_AP_RW_EL1;
    } else {
        pte |= PTE_AP_R0_EL1;
    }
    if !flags.executable {
        pte |= PTE_UXN | PTE_PXN;
    }
}
}

pte
}

fn va_l0_index(va: usize) -> usize { (va >>> 39) & 0xFF }
fn va_l1_index(va: usize) -> usize { (va >>> 30) & 0xFF }
fn va_l2_index(va: usize) -> usize { (va >>> 21) & 0xFF }
fn va_l3_index(va: usize) -> usize { (va >>> 12) & 0xFF }

unsafe fn get_or_create_table(table: &mut PageTable, index: usize) -> &mut PageTable {
    let entry = table.entries[index];
    if entry & PTE_VALID != 0 {
        let next_phys = (entry & 0x0000_FFFF_FFFF_F000) as usize;
        &mut *(next_phys as *mut PageTable)
    } else {
        let new_page = pmm::alloc_page().expect("OOM allocating page table");
        let ptr = new_page.as_usize() as *mut PageTable;
        table.entries[index] = new_page.as_usize() as u64 | PTE_VALID | PTE_TABLE;
        &mut *ptr
    }
}

pub unsafe fn map_page(l0: &mut PageTable, va: VirtAddr, pa: PhysAddr, flags: &MappingFlags) {
    let l1 = get_or_create_table(l0, va_l0_index(va.as_usize()));
    let l2 = get_or_create_table(l1, va_l1_index(va.as_usize()));
    let l3 = get_or_create_table(l2, va_l2_index(va.as_usize()));

    let pte = (pa.as_usize() as u64 & 0x0000_FFFF_FFFF_F000) | flags_to_pte(flags);
    l3.entries[va_l3_index(va.as_usize())] = pte;
}

// Initial boot page tables (statically allocated, used before PMM is ready)
#[repr(C, align(4096))]
struct BootPageTables {
    l0: PageTable,
    l1_low: PageTable, // Identity map for first 1GB
    l1_high: PageTable, // Higher-half map
    l2_low: [PageTable; 2],
    l2_high: [PageTable; 2],
}

static mut BOOT_TABLES: BootPageTables = BootPageTables {
    l0: PageTable { entries: [0; ENTRIES_PER_TABLE] },
    l1_low: PageTable { entries: [0; ENTRIES_PER_TABLE] },
    l1_high: PageTable { entries: [0; ENTRIES_PER_TABLE] },
    l2_low: [
        PageTable { entries: [0; ENTRIES_PER_TABLE] },

```



```

    PageTable { entries: [0; ENTRIES_PER_TABLE] },
],
l2_high: [
    PageTable { entries: [0; ENTRIES_PER_TABLE] },
    PageTable { entries: [0; ENTRIES_PER_TABLE] },
],
};

// Block descriptor flags (for 2MB blocks at L2)
const BLOCK_VALID: u64 = 1 &&&& 0;
const BLOCK_AF: u64 = 1 &&&& 10;
const BLOCK_SH_INNER: u64 = 3 &&&& 8;

pub unsafe fn init_boot_page_tables() {
    // We use 2MB block mappings at L2 level for simplicity during boot
    // This avoids needing L3 tables

    // L0[0] -&gt; l1_low (for identity map, VA 0x0000_0000_0000_0000 - 0x0000_007F_FFFF_FFFF)
    let l1_low_addr = &raw const BOOT_TABLES.l1_low as *const _ as u64;
    BOOT_TABLES.l0.entries[0] = l1_low_addr | PTE_VALID | PTE_TABLE;

    // L0[511] -&gt; l1_high (for higher-half, VA 0xFFFF_FF80_0000_0000 - 0xFFFF_FFFF_FFFF_FFFF)
    // Actually for KERNEL_VA_BASE = 0xFFFF_0000_0000_0000, L0 index = (0xFFFF_0000_0000_0000 &&&& 39) & 0x1FF
    let l0_high_idx = va_l0_index(KERNEL_VA_BASE);
    let l1_high_addr = &raw const BOOT_TABLES.l1_high as *const _ as u64;
    BOOT_TABLES.l0.entries[l0_high_idx] = l1_high_addr | PTE_VALID | PTE_TABLE;

    // Identity map: first 1GB as device memory (MMIO region 0x0000_0000 - 0x3FFF_FFFF)
    // L1_low[0] -&gt; l2_low[0] (first 512 * 2MB = 1GB)
    let l2_low0_addr = &raw const BOOT_TABLES.l2_low[0] as *const _ as u64;
    BOOT_TABLES.l1_low.entries[0] = l2_low0_addr | PTE_VALID | PTE_TABLE;

    // Map first 1GB as device-nGnRnE 2MB blocks
    for i in 0..512 {
        let phys = (i as u64) &&&& 21;
        BOOT_TABLES.l2_low[0].entries[i] = phys | BLOCK_VALID | BLOCK_AF
            | attr_index(MAIR_DEVICE_IDX);
    }

    // Identity map: RAM at 0x4000_0000 (L1_low[1])
    let l2_low1_addr = &raw const BOOT_TABLES.l2_low[1] as *const _ as u64;
    BOOT_TABLES.l1_low.entries[1] = l2_low1_addr | PTE_VALID | PTE_TABLE;

    // Map 1GB of RAM (0x4000_0000 - 0x7FFF_FFFF) as normal memory
    for i in 0..512 {
        let phys = 0x4000_0000 + ((i as u64) &&&& 21);
        BOOT_TABLES.l2_low[1].entries[i] = phys | BLOCK_VALID | BLOCK_AF
            | attr_index(MAIR_NORMAL_IDX) | BLOCK_SH_INNER;
    }

    // Higher-half: map MMIO (same as identity, at 0xFFFF_0000_0000_0000)
    // L1_high[0] -&gt; l2_high[0]
    let l2_high0_addr = &raw const BOOT_TABLES.l2_high[0] as *const _ as u64;
    BOOT_TABLES.l1_high.entries[0] = l2_high0_addr | PTE_VALID | PTE_TABLE;

    for i in 0..512 {
        let phys = (i as u64) &&&& 21;
        BOOT_TABLES.l2_high[0].entries[i] = phys | BLOCK_VALID | BLOCK_AF
            | attr_index(MAIR_DEVICE_IDX);
    }

    // Higher-half: map RAM at 0xFFFF_0000_4000_0000
    // L1 index for 0xFFFF_0000_4000_0000 = (0x4000_0000 &&&& 30) & 0x1FF = 1
    let l2_high1_addr = &raw const BOOT_TABLES.l2_high[1] as *const _ as u64;
    BOOT_TABLES.l1_high.entries[1] = l2_high1_addr | PTE_VALID | PTE_TABLE;
}

```

```

    for i in 0..512 {
        let phys = 0x4000_0000 + ((i as u64) &lt;&lt; 21);
        BOOT_TABLES.l2_high[1].entries[i] = phys | BLOCK_VALID | BLOCK_AF
            | attr_index(MAIR_NORMAL_IDX) | BLOCK_SH_INNER;
    }
}

pub unsafe fn enable_mmu() {
    let ttbr0 = &raw const BOOT_TABLES.l0 as *const _ as u64;
    let ttbr1 = &raw const BOOT_TABLES.l0 as *const _ as u64;

    // MAIR: index 0 = normal (write-back), index 1 = device-nGnRnE
    let mair: u64 = 0xFF | (0x00 &lt;&lt; 8);

    // TCR_EL1:
    // T0SZ = 16 (48-bit VA for TTBR0)
    // T1SZ = 16 (48-bit VA for TTBR1)
    // TG0 = 0b00 (4KB granule for TTBR0)
    // TG1 = 0b10 (4KB granule for TTBR1)
    // SH0 = SH1 = 0b11 (inner shareable)
    // ORGN0 = IRGN0 = ORGN1 = IRGN1 = 0b01 (write-back, write-allocate)
    // IPS = 0b010 (40-bit PA)
    let tcr: u64 = (16) // T0SZ
        | (16 &lt;&lt; 16) // T1SZ
        | (0b00 &lt;&lt; 14) // TG0
        | (0b10 &lt;&lt; 30) // TG1
        | (0b11 &lt;&lt; 12) // SH0
        | (0b11 &lt;&lt; 28) // SH1
        | (0b01 &lt;&lt; 10) // ORGN0
        | (0b01 &lt;&lt; 8) // IRGN0
        | (0b01 &lt;&lt; 26) // ORGN1
        | (0b01 &lt;&lt; 24) // IRGN1
        | (0b010 &lt;&lt; 32); // IPS (40-bit)

    core::arch::asm!(
        "msr MAIR_EL1, {mair}",
        "msr TCR_EL1, {tcr}",
        "msr TTBR0_EL1, {ttbr0}",
        "msr TTBR1_EL1, {ttbr1}",
        "isb",
        // Enable MMU + caches
        "mrs {tmp}, SCTLR_EL1",
        "orr {tmp}, {tmp}, #1", // M bit (MMU enable)
        "orr {tmp}, {tmp}, #(1 &lt;&lt; 2)", // C bit (data cache)
        "orr {tmp}, {tmp}, #(1 &lt;&lt; 12)", // I bit (instruction cache)
        "msr SCTLR_EL1, {tmp}",
        "isb",
        mair = in(reg) mair,
        tcr = in(reg) tcr,
        ttbr0 = in(reg) ttbr0,
        ttbr1 = in(reg) ttbr1,
        tmp = out(reg) _,
    );

    crate::println!("[vmm] MMU enabled with identity + higher-half mapping");
}

```

5.3 MMU Initialization

MMU setup configures MAIR_EL1 (index 0 = normal WB-WA 0xFF, index 1 = device-nGnRnE 0x00), TCR_EL1 (T0SZ=T1SZ=16 for 48-bit VA, 4KB granule, inner shareable, 40-bit IPS), and loads TTBR0/TTBR1 with the boot page table address. Both TTBR0 and TTBR1 initially point to the same L0

table, providing both identity-mapped and higher-half access.

kernel/src/mm/mod.rs

```
pub mod pmm;
pub mod vmm;
pub mod heap;

pub const PAGE_SIZE: usize = 4096;
pub const PAGE_SHIFT: usize = 12;

pub const KERNEL_VA_BASE: usize = 0xFFFF_0000_0000_0000;

#[derive(Copy, Clone, Debug, Eq, PartialEq, Ord, PartialOrd)]
#[repr(transparent)]
pub struct PhysAddr(pub usize);

#[derive(Copy, Clone, Debug, Eq, PartialEq, Ord, PartialOrd)]
#[repr(transparent)]
pub struct VirtAddr(pub usize);

impl PhysAddr {
    pub fn as_usize(self) -> usize {
        self.0
    }

    pub fn page_aligned(self) -> bool {
        self.0 & (PAGE_SIZE - 1) == 0
    }

    pub fn to_virt(self) -> VirtAddr {
        VirtAddr(self.0 + KERNEL_VA_BASE)
    }
}

impl VirtAddr {
    pub fn as_usize(self) -> usize {
        self.0
    }

    pub fn to_phys(self) -> PhysAddr {
        PhysAddr(self.0 - KERNEL_VA_BASE)
    }
}

pub fn init() {
    let ram_base = 0x4000_0000usize;
    let ram_size = 256 * 1024 * 1024; // 256 MB

    unsafe {
        pmm::init(ram_base, ram_size);
    }

    crate::println!(
        "[mm] Physical memory: {} pages free / {} total",
        pmm::free_pages(),
        pmm::total_pages()
    );
}
```

6. Phase 4: Kernel Heap Allocator

The heap allocator implements `GlobalAlloc` using a linked-list of free blocks. It is initialized with 1MB (256 pages) allocated from the PMM. The allocator uses first-fit allocation and coalesces adjacent free blocks on deallocation to reduce fragmentation. Thread safety is provided by an inline spin-lock mutex.

With the heap in place, the kernel gains access to Rust's `alloc` crate, enabling `Box`, `Vec`, `String`, `BTreeMap`, and other heap-allocated data structures.

kernel/src/mm/heap.rs

```
use core::alloc::{GlobalAlloc, Layout};
use core::ptr::null_mut;
use super::{pmm, PAGE_SIZE};

const HEAP_SIZE: usize = 1024 * 1024; // 1 MB initial heap

struct FreeBlock {
    size: usize,
    next: *mut FreeBlock,
}

const MIN_BLOCK_SIZE: usize = core::mem::size_of::<FreeBlock>();

pub struct LinkedListAllocator {
    head: *mut FreeBlock,
    initialized: bool,
}

unsafe impl Send for LinkedListAllocator {}
unsafe impl Sync for LinkedListAllocator {}

impl LinkedListAllocator {
    const fn new() -> Self {
        Self {
            head: null_mut(),
            initialized: false,
        }
    }

    unsafe fn init(&mut self) {
        let num_pages = HEAP_SIZE / PAGE_SIZE;
        let first_page = pmm::alloc_page().expect("OOM during heap init");
        let heap_start = first_page.as_usize();

        for _ in 1..num_pages {
            pmm::alloc_page().expect("OOM during heap init");
        }

        let block = heap_start as *mut FreeBlock;
        (*block).size = HEAP_SIZE;
        (*block).next = null_mut();
        self.head = block;
        self.initialized = true;

        crate::println!(
            "[heap] Initialized {} KB at {:#x}",
            HEAP_SIZE / 1024,
            heap_start
        );
    }
}
```

```

    });
}

fn align_up(addr: usize, align: usize) -> usize {
    (addr + align - 1) & !(align - 1)
}

unsafe fn alloc_inner(&mut self, layout: Layout) -> *mut u8 {
    let size = Self::align_up(layout.size().max(MIN_BLOCK_SIZE), layout.align());

    let mut prev: *mut FreeBlock = null_mut();
    let mut current = self.head;

    while !current.is_null() {
        let block_addr = current as usize;
        let aligned_addr = Self::align_up(block_addr, layout.align());
        let padding = aligned_addr - block_addr;
        let total_needed = padding + size;

        if (*current).size >= total_needed {
            let remaining = (*current).size - total_needed;

            if remaining >= MIN_BLOCK_SIZE {
                // Split: create a new free block after the allocated region
                let new_block = (aligned_addr + size) as *mut FreeBlock;
                (*new_block).size = remaining;
                (*new_block).next = (*current).next;

                if prev.is_null() {
                    self.head = new_block;
                } else {
                    (*prev).next = new_block;
                }
            } else {
                // Use entire block
                if prev.is_null() {
                    self.head = (*current).next;
                } else {
                    (*prev).next = (*current).next;
                }
            }
        }

        return aligned_addr as *mut u8;
    }

    prev = current;
    current = (*current).next;
}

null_mut()
}

unsafe fn dealloc_inner(&mut self, ptr: *mut u8, layout: Layout) {
    let size = Self::align_up(layout.size().max(MIN_BLOCK_SIZE), layout.align());
    let free_addr = ptr as usize;

    // Insert in address order and merge adjacent blocks
    let new_block = free_addr as *mut FreeBlock;
    (*new_block).size = size;
    (*new_block).next = null_mut();

    if self.head.is_null() || (new_block as usize) < (self.head as usize) {
        (*new_block).next = self.head;
        self.head = new_block;
    } else {

```

```

        let mut current = self.head;
        while !(*current).next.is_null()
            && ((*current).next as usize) < free_addr
        {
            current = (*current).next;
        }
        (*new_block).next = (*current).next;
        (*current).next = new_block;
    }

    // Merge with next block
    let block = new_block;
    if !(*block).next.is_null() {
        let block_end = (block as usize) + (*block).size;
        if block_end == (*block).next as usize {
            (*block).size += ((*block).next).size;
            (*block).next = ((*block).next).next;
        }
    }

    // Merge previous with current (re-scan from head to find prev)
    let mut current = self.head;
    while !(*current).next.is_null() {
        let current_end = (current as usize) + (*current).size;
        if current_end == (*current).next as usize {
            (*current).size += ((*current).next).size;
            (*current).next = ((*current).next).next;
        } else {
            current = (*current).next;
        }
    }
}

struct LockedAllocator {
    inner: spin::Mutex<LinkedListAllocator>;
}

impl LockedAllocator {
    const fn new() -> Self {
        Self {
            inner: spin::Mutex::new(LinkedListAllocator::new()),
        }
    }
}

unsafe impl GlobalAlloc for LockedAllocator {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let mut allocator = self.inner.lock();
        allocator.alloc_inner(layout)
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        let mut allocator = self.inner.lock();
        allocator.dealloc_inner(ptr, layout);
    }
}

#[global_allocator]
static ALLOCATOR: LockedAllocator = LockedAllocator::new();

pub fn init() {
    unsafe {
        ALLOCATOR.inner.lock().init();
    }
}

```

```

}

// Simple inline spin mutex to avoid external dependency
mod spin {
  use core::cell::UnsafeCell;
  use core::sync::atomic::{AtomicBool, Ordering};

  pub struct Mutex<T> {
    locked: AtomicBool,
    data: UnsafeCell<T>,
  }

  unsafe impl<T: Send> Send for Mutex<T> {}
  unsafe impl<T: Send> Sync for Mutex<T> {}

  impl<T> Mutex<T> {
    pub const fn new(data: T) -> Self {
      Self {
        locked: AtomicBool::new(false),
        data: UnsafeCell::new(data),
      }
    }

    pub fn lock(&self) -> MutexGuard<T> {
      while self
        .locked
        .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_err()
      {
        while self.locked.load(Ordering::Relaxed) {
          core::hint::spin_loop();
        }
      }
      MutexGuard { mutex: self }
    }
  }

  pub struct MutexGuard<'a, T> {
    mutex: &'a Mutex<T>,
  }

  impl<'a, T> core::ops::Deref for MutexGuard<'a, T> {
    type Target = T;
    fn deref(&self) -> &T {
      unsafe { &*self.mutex.data.get() }
    }
  }

  impl<'a, T> core::ops::DerefMut for MutexGuard<'a, T> {
    fn deref_mut(&mut self) -> &mut T {
      unsafe { &mut *self.mutex.data.get() }
    }
  }

  impl<'a, T> Drop for MutexGuard<'a, T> {
    fn drop(&mut self) {
      self.mutex.locked.store(false, Ordering::Release);
    }
  }
}

```

7. Phase 5: Synchronization & Preemptive Scheduler

7.1 SpinLock with Interrupt Masking

The SpinLock saves and disables interrupts (DAIF) on lock acquisition, restoring them on drop. This prevents deadlocks from timer IRQs attempting to acquire a lock already held. The lock uses AtomicBool with Acquire/Release ordering and a test-and-test-and-set pattern.

kernel/src/sync/spinlock.rs

```
use core::cell::UnsafeCell;
use core::sync::atomic::{AtomicBool, Ordering};

pub struct SpinLock<T> {
    locked: AtomicBool,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send> Send for SpinLock<T> {}
unsafe impl<T: Send> Sync for SpinLock<T> {}

impl<T> SpinLock<T> {
    pub const fn new(data: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            data: UnsafeCell::new(data),
        }
    }

    pub fn lock(&self) -> SpinLockGuard<T> {
        // Save and disable interrupts
        let daif: u64;
        unsafe {
            core::arch::asm!("mrs {}, DAIF", out(reg) daif);
            core::arch::asm!("msr DAIFSet, #0b1111");
        }

        while self
            .locked
            .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)
            .is_err()
        {
            while self.locked.load(Ordering::Relaxed) {
                core::hint::spin_loop();
            }
        }

        SpinLockGuard {
            lock: self,
            saved_daif: daif,
        }
    }
}

pub struct SpinLockGuard<'a, T> {
```



```

        lock: &'a SpinLock<T>;,
        saved_daif: u64,
    }

impl<'a, T> core::ops::Deref for SpinLockGuard<'a, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.lock.data.get() }
    }
}

impl<'a, T> core::ops::DerefMut for SpinLockGuard<'a, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }
    }
}

impl<'a, T> Drop for SpinLockGuard<'a, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
        // Restore interrupt state
        unsafe {
            core::arch::asm!("msr DAIF, {}", in(reg) self.saved_daif);
        }
    }
}

```

7.2 Task Structure & Context

Each task has a `TaskContext` struct (`repr(C)` for assembly interop) containing callee-saved registers x19-x30, SP, and state tracking. Tasks are allocated with 4-page (16KB) kernel stacks from the PMM.

kernel/src/sched/task.rs

```

use alloc::boxed::Box;
use crate::mm::{pmm, PAGE_SIZE};

const KERNEL_STACK_PAGES: usize = 4; // 16 KB per task

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum TaskState {
    Ready,
    Running,
    Dead,
}

#[repr(C)]
pub struct TaskContext {
    pub x19: u64,
    pub x20: u64,
    pub x21: u64,
    pub x22: u64,
    pub x23: u64,
    pub x24: u64,
    pub x25: u64,
    pub x26: u64,
    pub x27: u64,
    pub x28: u64,
    pub x29: u64,
    pub lr: u64,
    pub sp: u64,
}

```

```

pub type TaskId = u64;

pub struct Task {
    pub id: TaskId,
    pub state: TaskState,
    pub context: TaskContext,
    pub stack_base: usize,
    pub name: [u8; 32],
    pub name_len: usize,
}

impl Task {
    pub fn new(id: TaskId, name: &str, entry: fn() -> ! -> Box<Self> {
        // Allocate kernel stack
        let mut stack_base = 0usize;
        for i in 0..KERNEL_STACK_PAGES {
            let page = pmm::alloc_page().expect("OOM allocating task stack");
            if i == 0 {
                stack_base = page.as_usize();
            }
        }

        let stack_top = stack_base + KERNEL_STACK_PAGES * PAGE_SIZE;

        let mut name_buf = [0u8; 32];
        let len = name.len().min(31);
        name_buf[..len].copy_from_slice(&name.as_bytes()[..len]);

        Box::new(Self {
            id,
            state: TaskState::Ready,
            context: TaskContext {
                x19: 0, x20: 0, x21: 0, x22: 0,
                x23: 0, x24: 0, x25: 0, x26: 0,
                x27: 0, x28: 0,
                x29: 0,
                lr: entry as u64,
                sp: stack_top as u64,
            },
            stack_base,
            name: name_buf,
            name_len: len,
        })
    }

    pub fn new_idle(id: TaskId) -> Box<Self> {
        Box::new(Self {
            id,
            state: TaskState::Running,
            context: TaskContext {
                x19: 0, x20: 0, x21: 0, x22: 0,
                x23: 0, x24: 0, x25: 0, x26: 0,
                x27: 0, x28: 0, x29: 0,
                lr: 0, sp: 0,
            },
            stack_base: 0,
            name: {
                let mut n = [0u8; 32];
                n[..4].copy_from_slice(b"idle");
                n
            },
            name_len: 4,
        })
    }
}

```

```

    pub fn name_str(&self) -> &str {
        core::str::from_utf8(&self.name[..self.name_len]).unwrap_or("???")
    }
}

```

7.3 Round-Robin Scheduler

The scheduler maintains a VecDeque of tasks. On each timer tick, `schedule()` is called: it moves the current task to the back of the queue, picks the next Ready task, and performs a context switch. A `task_entry_trampoline` in assembly enables interrupts (DAIFCIR) before calling the task's entry function, bridging the IRQ-disabled context switch to normal execution.

kernel/src/sched/scheduler.rs

```

use alloc::boxed::Box;
use alloc::collections::VecDeque;
use core::arch::global_asm;
use core::sync::atomic::{AtomicBool, Ordering};
use super::task::{Task, TaskState, TaskId, TaskContext};

struct SchedulerInner {
    tasks: VecDeque<Box<Task>>,
    current: Option<Box<Task>>,
    next_id: TaskId,
}

static mut SCHEDULER: Option<SchedulerInner> = None;
static SCHEDULER_ACTIVE: AtomicBool = AtomicBool::new(false);

global_asm!(
    "
    .global context_switch
    context_switch:
        // x0 = prev TaskContext*, x1 = next TaskContext*

        stp    x19, x20, [x0, #0]
        stp    x21, x22, [x0, #16]
        stp    x23, x24, [x0, #32]
        stp    x25, x26, [x0, #48]
        stp    x27, x28, [x0, #64]
        stp    x29, x30, [x0, #80]
        mov    x2, sp
        str    x2, [x0, #96]

        ldp    x19, x20, [x1, #0]
        ldp    x21, x22, [x1, #16]
        ldp    x23, x24, [x1, #32]
        ldp    x25, x26, [x1, #48]
        ldp    x27, x28, [x1, #64]
        ldp    x29, x30, [x1, #80]
        ldr    x2, [x1, #96]
        mov    sp, x2

        ret
    "
);

extern "C" {
    fn context_switch(prev: *mut TaskContext, next: *const TaskContext);
}

// Trampoline for new tasks: enables interrupts then calls the real entry point

```

```

// The entry point address is passed in x19 (callee-saved, survives context_switch)
global_asm!(
    "
        .global task_entry_trampoline
        task_entry_trampoline:
            msr    DAIFClr, #0b0010
            mov    x0, x19
            blr    x0
            b      .
    "
);

extern "C" {
    fn task_entry_trampoline();
}

pub fn init() {
    unsafe {
        SCHEDULER = Some(SchedulerInner {
            tasks: VecDeque::new(),
            current: Some(Task::new_idle(0)),
            next_id: 1,
        });
    }
    SCHEDULER_ACTIVE.store(true, Ordering::Release);
    crate::println!("[sched] Scheduler initialized");
}

pub fn spawn(name: &str, entry: fn() -&gt; !) {
    unsafe {
        let sched = SCHEDULER.as_mut().unwrap();
        let id = sched.next_id;
        sched.next_id += 1;
        let mut task = Task::new(id, name, entry);
        // Set x19 to the real entry point, lr to the trampoline
        task.context.x19 = entry as u64;
        task.context.lr = task_entry_trampoline as u64;
        crate::println!("[sched] Spawned task {} (id={})", name, id);
        sched.tasks.push_back(task);
    }
}

pub fn schedule() {
    if !SCHEDULER_ACTIVE.load(Ordering::Relaxed) {
        return;
    }

    unsafe {
        // Disable interrupts during scheduling
        core::arch::asm!("msr DAIFSet, #0b1111");

        let sched = match SCHEDULER.as_mut() {
            Some(s) => s,
            None => {
                core::arch::asm!("msr DAIFClr, #0b0010");
                return;
            }
        };

        if sched.tasks.is_empty() {
            core::arch::asm!("msr DAIFClr, #0b0010");
            return;
        }

        let next = match sched.tasks.pop_front() {

```

```

        Some(t) => t,
        None => {
            core::arch::asm!("msr DAIFClr, #0b0010");
            return;
        }
    };

    let mut prev = sched.current.take().unwrap();
    if prev.state != TaskState::Dead {
        prev.state = TaskState::Ready;
        sched.tasks.push_back(prev);
    }

    let prev_task = sched.tasks.back_mut().unwrap();
    sched.current = Some(next);
    let current = sched.current.as_mut().unwrap();
    current.state = TaskState::Running;

    let prev_ctx = &mut prev_task.context as *mut TaskContext;
    let next_ctx = &mut current.context as *const TaskContext;

    context_switch(prev_ctx, next_ctx);

    // After switching back to this task, re-enable interrupts
    core::arch::asm!("msr DAIFClr, #0b0010");
}

pub fn current_task_name() -> [u8; 32] {
    unsafe {
        match SCHEDULER.as_ref() {
            Some(sched) => match &sched.current {
                Some(t) => t.name,
                None => [0u8; 32],
            },
            None => [0u8; 32],
        }
    }
}

```

7.4 Assembly Context Switch

The `context_switch` function saves callee-saved registers (x19-x30) and SP to the previous task's `TaskContext`, then loads them from the next task's `TaskContext` and returns via the loaded LR (x30). This seamlessly resumes the next task from wherever it was last preempted.

8. Phase 6: Device Drivers

8.1 VirtIO MMIO Transport

VirtIO devices are discovered by probing 32 MMIO slots starting at 0x0A00_0000 with a stride of 0x200. Each slot is checked for the magic value 0x74726976 ('virt' in little-endian). Device initialization follows the VirtIO specification: reset, ACKNOWLEDGE, DRIVER, feature negotiation, FEATURES_OK, queue setup, and DRIVER_OK. The driver supports both legacy (v1) and modern (v2) transports.

kernel/src/drivers/virtio/mod.rs

```
pub mod queue;
pub mod block;
pub mod net;

use core::ptr::{read_volatile, write_volatile};

const VIRTIO_MMIO_BASE: usize = 0x0A00_0000;
const VIRTIO_MMIO_STRIDE: usize = 0x200;
const VIRTIO_MMIO_COUNT: usize = 32;

// MMIO register offsets (common to v1 and v2)
const MAGIC: usize = 0x000;
const VERSION: usize = 0x004;
const DEVICE_ID: usize = 0x008;
const DEVICE_FEATURES: usize = 0x010;
const DEVICE_FEATURES_SEL: usize = 0x014;
const DRIVER_FEATURES: usize = 0x020;
const DRIVER_FEATURES_SEL: usize = 0x024;
const GUEST_PAGE_SIZE: usize = 0x028; // v1 only
const QUEUE_SEL: usize = 0x030;
const QUEUE_NUM_MAX: usize = 0x034;
const QUEUE_NUM: usize = 0x038;
const QUEUE_ALIGN: usize = 0x03C; // v1 only
const QUEUE_PFN: usize = 0x040; // v1 only
const QUEUE_READY: usize = 0x044; // v2 only
const QUEUE_NOTIFY: usize = 0x050;
const INTERRUPT_STATUS: usize = 0x060;
const INTERRUPT_ACK: usize = 0x064;
const STATUS: usize = 0x070;
// v2 only registers
const QUEUE_DESC_LOW: usize = 0x080;
const QUEUE_DESC_HIGH: usize = 0x084;
const QUEUE_AVAIL_LOW: usize = 0x090;
const QUEUE_AVAIL_HIGH: usize = 0x094;
const QUEUE_USED_LOW: usize = 0x0A0;
const QUEUE_USED_HIGH: usize = 0x0A4;

const DEVICE_NET: u32 = 1;
const DEVICE_BLOCK: u32 = 2;

const STATUS_ACKNOWLEDGE: u32 = 1;
const STATUS_DRIVER: u32 = 2;
const STATUS_FEATURES_OK: u32 = 8;
const STATUS_DRIVER_OK: u32 = 4;

const VIRTIO_MAGIC: u32 = 0x7472_6976;
```

```

pub struct VirtioMmio {
    pub base: usize,
    pub device_id: u32,
    pub version: u32,
}

impl VirtioMmio {
    fn read(&self, offset: usize) -> u32 {
        unsafe { read_volatile((self.base + offset) as *const u32) }
    }

    fn write(&self, offset: usize, val: u32) {
        unsafe { write_volatile((self.base + offset) as *mut u32, val) }
    }

    pub fn init_device(&self) -> bool {
        // Reset
        self.write(STATUS, 0);

        // Acknowledge
        self.write(STATUS, STATUS_ACKNOWLEDGE);
        self.write(STATUS, STATUS_ACKNOWLEDGE | STATUS_DRIVER);

        if self.version == 1 {
            // Legacy: set guest page size, read features directly
            self.write(GUEST_PAGE_SIZE, 4096);
            let features = self.read(DEVICE_FEATURES);
            self.write(DRIVER_FEATURES, features);
            self.write(STATUS, STATUS_ACKNOWLEDGE | STATUS_DRIVER | STATUS_DRIVER_OK);
        } else {
            // Modern: use FEATURES_SEL
            self.write(DEVICE_FEATURES_SEL, 0);
            let features = self.read(DEVICE_FEATURES);
            self.write(DRIVER_FEATURES_SEL, 0);
            self.write(DRIVER_FEATURES, features);
            self.write(STATUS, STATUS_ACKNOWLEDGE | STATUS_DRIVER | STATUS_FEATURES_OK);
            let status = self.read(STATUS);
            if status & STATUS_FEATURES_OK == 0 {
                crate::println!("[virtio] Features negotiation failed");
                return false;
            }
        }

        true
    }

    pub fn setup_queue(&self, queue_idx: u32, vq: &queue::Virtqueue) {
        self.write(Queue_SEL, queue_idx);
        self.write(Queue_NUM, vq.size as u32);

        if self.version == 1 {
            // Legacy: set Queue_ALIGN and Queue_PFN
            self.write(Queue_ALIGN, 4096);
            let pfn = vq.desc_addr() as u32 / 4096;
            self.write(Queue_PFN, pfn);
        } else {
            let desc_addr = vq.desc_addr();
            self.write(Queue_DESC_LOW, desc_addr as u32);
            self.write(Queue_DESC_HIGH, (desc_addr >>> 32) as u32);

            let avail_addr = vq.avail_addr();
            self.write(Queue_AVAIL_LOW, avail_addr as u32);
            self.write(Queue_AVAIL_HIGH, (avail_addr >>> 32) as u32);

            let used_addr = vq.used_addr();

```

```

        self.write(QueueUsedLow, used_addr as u32);
        self.write(QueueUsedHigh, (used_addr >> 32) as u32);

        self.write(QueueReady, 1);
    }
}

pub fn driver_ok(&self) {
    let status = self.read(STATUS);
    self.write(STATUS, status | STATUS_DRIVER_OK);
}

pub fn notify(&self, queue_idx: u32) {
    self.write(QueueNotify, queue_idx);
}

pub fn queue_num_max(&self, queue_idx: u32) -> u32 {
    self.write(QueueSel, queue_idx);
    self.read(QueueNumMax)
}

pub fn ack_interrupt(&self) {
    let status = self.read(INTERRUPT_STATUS);
    self.write(INTERRUPT_ACK, status);
}
}

pub fn probe() -> alloc::vec::Vec<VirtioMmio> {
    let mut devices = alloc::vec::Vec::new();

    for i in 0..VIRTIO_MMIO_COUNT {
        let base = VIRTIO_MMIO_BASE + i * VIRTIO_MMIO_STRIDE;
        let magic = unsafe { read_volatile(base as *const u32) };
        if magic != VIRTIO_MAGIC {
            continue;
        }

        let device_id = unsafe { read_volatile((base + DEVICE_ID) as *const u32) };
        if device_id == 0 {
            continue;
        }

        let version = unsafe { read_volatile((base + VERSION) as *const u32) };
        let name = match device_id {
            DEVICE_NET => "network",
            DEVICE_BLOCK => "block",
            _ => "unknown",
        };

        crate::println!(
            "[virtio] Found {} device (id={}) at {:#x}, version {}",
            name, device_id, base, version
        );

        devices.push(VirtioMmio { base, device_id, version });
    }

    devices
}

```

8.2 Split Virtqueue Implementation

The split virtqueue consists of three components: a descriptor table (buffer addresses and sizes), an available ring (descriptors offered to the device), and a used ring (descriptors returned by the device). The legacy layout requires the used ring to be page-aligned after the descriptor table and available ring. Descriptors are managed as a free list.

kernel/src/drivers/virtio/queue.rs

```
use crate::mm::{pmm, PAGE_SIZE};

pub const QUEUE_SIZE: usize = 128;

#[repr(C)]
#[derive(Clone, Copy, Default)]
pub struct VirtqDesc {
    pub addr: u64,
    pub len: u32,
    pub flags: u16,
    pub next: u16,
}

pub const VRING_DESC_F_NEXT: u16 = 1;
pub const VRING_DESC_F_WRITE: u16 = 2;

#[repr(C)]
pub struct VirtqAvail {
    pub flags: u16,
    pub idx: u16,
    pub ring: [u16; QUEUE_SIZE],
}

#[repr(C)]
#[derive(Clone, Copy, Default)]
pub struct VirtqUsedElem {
    pub id: u32,
    pub len: u32,
}

#[repr(C)]
pub struct VirtqUsed {
    pub flags: u16,
    pub idx: u16,
    pub ring: [VirtqUsedElem; QUEUE_SIZE],
}

pub struct Virtqueue {
    pub desc: *mut VirtqDesc,
    pub avail: *mut VirtqAvail,
    pub used: *mut VirtqUsed,
    pub size: usize,
    pub free_head: u16,
    pub num_free: u16,
    pub last_used_idx: u16,
}

unsafe impl Send for Virtqueue {}

impl Virtqueue {
    /// Creates a legacy-compatible virtqueue with contiguous memory layout:
    /// [descriptors][available ring][padding to 4096][used ring]
    pub fn new(size: usize) -> Self {
        let desc_size = size * core::mem::size_of::<VirtqDesc>();
        let avail_size = 6 + 2 * size; // flags(2) + idx(2) + ring(2*size) + used_event(2)

        // Legacy layout: used ring must be page-aligned after desc+avail
```

```

let used_offset = align_up(desc_size + avail_size, 4096);
let used_size = 6 + 8 * size; // flags(2) + idx(2) + ring(8*size) + avail_event(2)

let total = used_offset + used_size;
let pages_needed = (total + PAGE_SIZE - 1) / PAGE_SIZE;

let first_page = pmm::alloc_page().expect("OOM allocating virtqueue");
let base = first_page.as_usize();
for _ in 1..pages_needed {
    pmm::alloc_page().expect("OOM allocating virtqueue");
}

let desc = base as *mut VirtqDesc;
let avail = (base + desc_size) as *mut VirtqAvail;
let used = (base + used_offset) as *mut VirtqUsed;

// Initialize free list
unsafe {
    for i in 0..size {
        let d = &mut *desc.add(i);
        d.addr = 0;
        d.len = 0;
        d.flags = 0;
        d.next = if i + 1 < size { (i + 1) as u16 } else { 0 };
    }

    let a = &mut *avail;
    a.flags = 0;
    a.idx = 0;

    let u = &mut *used;
    u.flags = 0;
    u.idx = 0;
}

Self {
    desc,
    avail,
    used,
    size,
    free_head: 0,
    num_free: size as u16,
    last_used_idx: 0,
}
}

pub fn desc_addr(&self) -> u64 {
    self.desc as u64
}

pub fn avail_addr(&self) -> u64 {
    self.avail as u64
}

pub fn used_addr(&self) -> u64 {
    self.used as u64
}

pub fn alloc_desc(&mut self) -> Option<u16> {
    if self.num_free == 0 {
        return None;
    }
    let idx = self.free_head;
    unsafe {
        self.free_head = (*self.desc.add(idx as usize)).next;
    }
}

```

```

    }
    self.num_free -= 1;
    Some(idx)
}

pub fn free_desc(&mut self, idx: u16) {
    unsafe {
        let d = &mut *self.desc.add(idx as usize);
        d.addr = 0;
        d.len = 0;
        d.flags = 0;
        d.next = self.free_head;
    }
    self.free_head = idx;
    self.num_free += 1;
}

pub fn push_avail(&mut self, desc_idx: u16) {
    unsafe {
        let a = &mut *self.avail;
        let idx = a.idx as usize % self.size;
        a ring[idx] = desc_idx;
        core::sync::atomic::fence(core::sync::atomic::Ordering::SeqCst);
        a.idx = a.idx.wrapping_add(1);
    }
}

pub fn pop_used(&mut self) -> Option<(u32, u32)> {
    unsafe {
        let u = &*self.used;
        if self.last_used_idx == u.idx {
            return None;
        }
        core::sync::atomic::fence(core::sync::atomic::Ordering::SeqCst);
        let idx = self.last_used_idx as usize % self.size;
        let elem = u ring[idx];
        self.last_used_idx = self.last_used_idx.wrapping_add(1);
        Some((elem.id, elem.len))
    }
}

fn align_up(val: usize, align: usize) -> usize {
    (val + align - 1) & !(align - 1)
}

```

8.3 VirtIO Block Driver

The block driver reads/writes 512-byte sectors using 3-descriptor chains: a request header (type + sector number), a data buffer, and a status byte. It polls the used ring for completion. This driver enables the FAT32 filesystem.

kernel/src/drivers/virtio/block.rs

```

use super::{VirtioMmio, queue::{Virtqueue, VRING_DESC_F_NEXT, VRING_DESC_F_WRITE}};
use crate::mm::pmm;

const VIRTIO_BLK_T_IN: u32 = 0;
const VIRTIO_BLK_T_OUT: u32 = 1;

#[repr(C)]
struct VirtioBlkReqHeader {
    req_type: u32,

```

```

        reserved: u32,
        sector: u64,
    }

pub struct VirtioBlock {
    mmio: VirtioMmio,
    queue: Virtqueue,
    req_buf: usize, // Physical address of request buffer page
}

impl VirtioBlock {
    pub fn new(mmio: VirtioMmio) -> Option<Self> {
        if !mmio.init_device() {
            return None;
        }

        let max_queue = mmio.queue_num_max(0);
        let queue_size = max_queue.min(128) as usize;
        let mut queue = Virtqueue::new(queue_size);
        mmio.setup_queue(0, &queue);
        mmio.driver_ok();

        // Allocate a buffer page for requests
        let req_buf = pmm::alloc_page().expect("OOM for block request buffer").as_usize();

        crate::println!("[virtio-blk] Initialized, queue size = {}", queue_size);

        Some(Self { mmio, queue, req_buf })
    }

    pub fn read_sector(&mut self, sector: u64, buf: &mut [u8; 512]) -> bool {
        self.do_request(VIRTIO_BLK_T_IN, sector, buf)
    }

    pub fn write_sector(&mut self, sector: u64, buf: &[u8; 512]) -> bool {
        let mut data = *buf;
        self.do_request(VIRTIO_BLK_T_OUT, sector, &mut data)
    }

    fn do_request(&mut self, req_type: u32, sector: u64, buf: &mut [u8; 512]) -> bool {
        // Layout in req_buf page:
        // offset 0: VirtioBlkReqHeader (16 bytes)
        // offset 16: data buffer (512 bytes)
        // offset 528: status byte
        let header_addr = self.req_buf;
        let data_addr = self.req_buf + 16;
        let status_addr = self.req_buf + 528;

        unsafe {
            let header = header_addr as *mut VirtioBlkReqHeader;
            (*header).req_type = req_type;
            (*header).reserved = 0;
            (*header).sector = sector;

            // For write, copy data to buffer
            if req_type == VIRTIO_BLK_T_OUT {
                core::ptr::copy_nonoverlapping(
                    buf.as_ptr(),
                    data_addr as *mut u8,
                    512,
                );
            }

            // Set status to 0xFF (sentinel)
            *(status_addr as *mut u8) = 0xFF;
        }
    }
}

```

```

    }

    // Build descriptor chain: header -> data -> status
    let d0 = self.queue.alloc_desc().expect("no free desc");
    let d1 = self.queue.alloc_desc().expect("no free desc");
    let d2 = self.queue.alloc_desc().expect("no free desc");

    unsafe {
        let desc0 = &mut *self.queue.desc.add(d0 as usize);
        desc0.addr = header_addr as u64;
        desc0.len = 16;
        desc0.flags = VRING_DESC_F_NEXT;
        desc0.next = d1;

        let desc1 = &mut *self.queue.desc.add(d1 as usize);
        desc1.addr = data_addr as u64;
        desc1.len = 512;
        desc1.flags = VRING_DESC_F_NEXT
            | if req_type == VIRTIO_BLK_T_IN { VRING_DESC_F_WRITE } else { 0 };
        desc1.next = d2;

        let desc2 = &mut *self.queue.desc.add(d2 as usize);
        desc2.addr = status_addr as u64;
        desc2.len = 1;
        desc2.flags = VRING_DESC_F_WRITE;
        desc2.next = 0;
    }

    self.queue.push_avail(d0);
    self.mmio.notify(0);

    // Poll for completion
    loop {
        if let Some((id, _len)) = self.queue.pop_used() {
            self.queue.free_desc(d2);
            self.queue.free_desc(d1);
            self.queue.free_desc(d0);

            let status = unsafe { *(status_addr as *const u8) };
            if req_type == VIRTIO_BLK_T_IN && status == 0 {
                unsafe {
                    core::ptr::copy_nonoverlapping(
                        data_addr as *const u8,
                        buf.as_mut_ptr(),
                        512,
                    );
                }
            }
            return status == 0;
        }
        core::hint::spin_loop();
    }
}
}

```

8.4 VirtIO Network Driver

The network driver manages RX and TX virtqueues. RX buffers are pre-populated and submitted to the device. Each frame is prefixed with a VirtioNetHeader (10 bytes). The MAC address is read from device configuration space at offset 0x100.

kernel/src/drivers/virtio/net.rs

```

use super::{VirtioMmio, queue::{Virtqueue, VRING_DESC_F_NEXT, VRING_DESC_F_WRITE, QUEUE_SIZE}};
use crate::mm::pmm;
use alloc::vec::Vec;

const VIRTIO_NET_HDR_SIZE: usize = 10;

#[repr(C)]
#[derive(Clone, Copy, Default)]
pub struct VirtioNetHeader {
    pub flags: u8,
    pub gso_type: u8,
    pub hdr_len: u16,
    pub gso_size: u16,
    pub csum_start: u16,
    pub csum_offset: u16,
}

const RX_BUF_SIZE: usize = 1526; // ETH_FRAME_MAX + VirtioNetHeader

pub struct VirtioNet {
    mmio: VirtioMmio,
    rx_queue: Virtqueue,
    tx_queue: Virtqueue,
    rx_buffers: Vec<usize>, // Physical addresses of RX buffer pages
    tx_buf: usize,           // Physical address of TX buffer page
    pub mac: [u8; 6],
}

impl VirtioNet {
    pub fn new(mmio: VirtioMmio) -> Option<Self> {
        if !mmio.init_device() {
            return None;
        }

        // Read MAC address from device config (offset 0x100)
        let mut mac = [0u8; 6];
        for i in 0..6 {
            mac[i] = unsafe {
                core::ptr::read_volatile((mmio.base + 0x100 + i) as *const u8)
            };
        }

        let rx_max = mmio.queue_num_max(0).min(128) as usize;
        let tx_max = mmio.queue_num_max(1).min(128) as usize;

        let mut rx_queue = Virtqueue::new(rx_max);
        let mut tx_queue = Virtqueue::new(tx_max);

        mmio.setup_queue(0, &rx_queue);
        mmio.setup_queue(1, &tx_queue);
        mmio.driver_ok();

        // Allocate RX buffers and populate RX queue
        let num_rx_bufs = rx_max.min(32);
        let mut rx_buffers = Vec::with_capacity(num_rx_bufs);

        for _ in 0..num_rx_bufs {
            let page = pmm::alloc_page().expect("OOM for net RX buffer");
            let buf_addr = page.as_usize();
            rx_buffers.push(buf_addr);

            if let Some(desc_idx) = rx_queue.alloc_desc() {
                unsafe {
                    let d = &mut *rx_queue.desc.add(desc_idx as usize);
                    d.addr = buf_addr as u64;
                }
            }
        }
    }
}

```

```

        d.len = RX_BUF_SIZE as u32;
        d.flags = VRING_DESC_F_WRITE;
        d.next = 0;
    }
    rx_queue.push_avail(desc_idx);
}

mmio.notify(0); // Notify device that RX buffers are ready

// Allocate TX buffer
let tx_buf = pmm::alloc_page().expect("OOM for net TX buffer").as_usize();

crate::println!(
    "[virtio-net] Initialized, MAC {:02x}:{:02x}:{:02x}:{:02x}:{:02x}:{:02x}",
    mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]
);

Some(Self {
    mmio,
    rx_queue,
    tx_queue,
    rx_buffers,
    tx_buf,
    mac,
})
}

pub fn send(&mut self, frame: &[u8]) -> bool {
    if frame.len() + VIRTIO_NET_HDR_SIZE > 4096 {
        return false;
    }

    // Write VirtioNetHeader + frame to TX buffer
    unsafe {
        let header = self.tx_buf as *mut VirtioNetHeader;
        *header = VirtioNetHeader::default();

        core::ptr::copy_nonoverlapping(
            frame.as_ptr(),
            (self.tx_buf + VIRTIO_NET_HDR_SIZE) as *mut u8,
            frame.len(),
        );
    }

    let desc_idx = match self.tx_queue.alloc_desc() {
        Some(idx) => idx,
        None => return false,
    };

    unsafe {
        let d = &mut *self.tx_queue.desc.add(desc_idx as usize);
        d.addr = self.tx_buf as u64;
        d.len = (VIRTIO_NET_HDR_SIZE + frame.len()) as u32;
        d.flags = 0;
        d.next = 0;
    }

    self.tx_queue.push_avail(desc_idx);
    self.mmio.notify(1);

    // Poll for completion
    loop {
        if let Some((id, _)) = self.tx_queue.pop_used() {
            self.tx_queue.free_desc(id as u16);

```

```

        return true;
    }
    core::hint::spin_loop();
}

pub fn recv(&mut self) -> Option<Vec<u8>> {
    let (id, len) = self.rx_queue.pop_used()?;

    let buf_addr = unsafe { (*self.rx_queue.desc.add(id as usize)).addr as usize };

    // Skip VirtioNetHeader, copy the Ethernet frame
    let frame_len = len as usize - VIRTIO_NET_HDR_SIZE;
    let mut frame = alloc::vec![0u8; frame_len];
    unsafe {
        core::ptr::copy_nonoverlapping(
            (buf_addr + VIRTIO_NET_HDR_SIZE) as *const u8,
            frame.as_mut_ptr(),
            frame_len,
        );
    }

    // Re-submit the buffer to RX queue
    unsafe {
        let d = &mut *self.rx_queue.desc.add(id as usize);
        d.addr = buf_addr as u64;
        d.len = RX_BUF_SIZE as u32;
        d.flags = VRING_DESC_F_WRITE;
        d.next = 0;
    }
    self.rx_queue.push_avail(id as u16);
    self.mmio.notify(0);

    Some(frame)
}
}

```

9. Phase 7: Filesystem

9.1 Virtual File System (VFS)

The VFS provides a unified interface for all filesystems through the `FileSystem` and `FileOps` traits. Mount points are stored in a sorted vector (longest path first) enabling longest-prefix matching. The VFS is protected by a `SpinLock` for thread safety.

kernel/src/fs/vfs.rs

```
extern crate alloc;

use alloc::boxed::Box;
use alloc::format;
use alloc::string::String;
use alloc::vec::Vec;
use crate::sync::spinlock::SpinLock;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum FileType {
    File,
    Directory,
}

#[derive(Debug, Clone)]
pub struct DirEntry {
    pub name: String,
    pub file_type: FileType,
    pub size: usize,
}

#[derive(Debug, Clone)]
pub struct Stat {
    pub file_type: FileType,
    pub size: usize,
}

pub trait FileOps: Send {
    fn read(&mut self, offset: usize, buf: &mut [u8]) -> Result<usize, &'static str>;
    fn write(&mut self, offset: usize, data: &[u8]) -> Result<usize, &'static str>;
    fn stat(&self) -> Result<Stat, &'static str>;
    fn close(&mut self) {}
}

pub trait FileSystem: Send {
    fn name(&self) -> &str;
    fn open(&self, path: &str) -> Result<Box<dyn FileOps>, &'static str>;
    fn readdir(&self, path: &str) -> Result<Vec<DirEntry>, &'static str>;
    fn stat(&self, path: &str) -> Result<Stat, &'static str>;
    fn create(&self, path: &str, file_type: FileType) -> Result<(), &'static str> {
        let _ = (path, file_type);
        Err("read-only filesystem")
    }
    fn remove(&self, path: &str) -> Result<(), &'static str> {
        let _ = path;
        Err("read-only filesystem")
    }
}
```

```

struct MountPoint {
    path: String,
    fs: Box<dyn FileSystem>;,
}

pub struct VFS {
    mounts: Vec<MountPoint>;,
}

static VFS_INSTANCE: SpinLock<Option<VFS>>; = SpinLock::new(None);

impl VFS {
    fn new() -> Self {
        Self {
            mounts: Vec::new(),
        }
    }

    fn mount_inner(&mut self, path: &str, fs: Box<dyn FileSystem>,) {
        let path = normalize_mount_path(path);
        crate::println!("[vfs] Mounting '{}' at {}", fs.name(), path);
        if let Some(existing) = self.mounts.iter().position(|m| m.path == path) {
            self.mounts.remove(existing);
        }
        self.mounts.push(MountPoint {
            path,
            fs,
        });
        self.mounts.sort_by(|a, b| b.path.len().cmp(&a.path.len()));
    }

    fn resolve(&self, path: &str) -> Option<(&dyn FileSystem, &str)> {
        let path = if path.is_empty() { "/" } else { path };
        for mount in &self.mounts {
            if mount.path == "/" {
                let relative = path.strip_prefix('/').unwrap_or(path);
                return Some((mount.fs.as_ref(), relative));
            }
            if path == mount.path || path.starts_with(&format!("{}", mount.path)) {
                let relative = &path[mount.path.len()..];
                let relative = relative.strip_prefix('/').unwrap_or(relative);
                return Some((mount.fs.as_ref(), relative));
            }
        }
        None
    }
}

fn normalize_mount_path(path: &str) -> String {
    if path == "/" {
        return String::from("/");
    }
    let p = path.trim_end_matches('/');
    if p.is_empty() {
        String::from("/")
    } else {
        String::from(p)
    }
}

pub fn init() {
    let mut vfs = VFS_INSTANCE.lock();
    *vfs = Some(VFS::new());
}

```

```

pub fn mount(path: &str, fs: Box<dyn FileSystem>) {
    let mut vfs = VFS_INSTANCE.lock();
    if let Some(ref mut v) = *vfs {
        v.mount_inner(path, fs);
    }
}

pub fn open(path: &str) -> Result<Box<dyn FileOps>, &'static str>; {
    let vfs = VFS_INSTANCE.lock();
    let v = vfs.as_ref().ok_or("VFS not initialized")?;
    let (fs, relative) = v.resolve(path).ok_or("no filesystem mounted for path")?;
    fs.open(relative)
}

pub fn readdir(path: &str) -> Result<Vec<DirEntry>, &'static str>; {
    let vfs = VFS_INSTANCE.lock();
    let v = vfs.as_ref().ok_or("VFS not initialized")?;
    let (fs, relative) = v.resolve(path).ok_or("no filesystem mounted for path")?;
    fs.readdir(relative)
}

pub fn stat(path: &str) -> Result<Stat, &'static str>; {
    let vfs = VFS_INSTANCE.lock();
    let v = vfs.as_ref().ok_or("VFS not initialized")?;
    let (fs, relative) = v.resolve(path).ok_or("no filesystem mounted for path")?;
    fs.stat(relative)
}

pub fn create(path: &str, file_type: FileType) -> Result<(), &'static str>; {
    let vfs = VFS_INSTANCE.lock();
    let v = vfs.as_ref().ok_or("VFS not initialized")?;
    let (fs, relative) = v.resolve(path).ok_or("no filesystem mounted for path")?;
    fs.create(relative, file_type)
}

pub fn remove(path: &str) -> Result<(), &'static str>; {
    let vfs = VFS_INSTANCE.lock();
    let v = vfs.as_ref().ok_or("VFS not initialized")?;
    let (fs, relative) = v.resolve(path).ok_or("no filesystem mounted for path")?;
    fs.remove(relative)
}

```

9.2 FAT32 Read-Only Driver

The FAT32 driver parses the BIOS Parameter Block (BPB) from sector 0, follows cluster chains through the FAT, and parses both 8.3 short directory entries and Long File Name (LFN) entries. Files are read by following the cluster chain and concatenating sectors. The block device is wrapped in a SpinLock for safe concurrent access.

kernel/src/fs/fat32.rs

```

extern crate alloc;

use alloc::boxed::Box;
use alloc::string::String;
use alloc::vec::Vec;
use crate::drivers::virtio::block::VirtioBlock;
use crate::sync::spinlock::SpinLock;
use super::vfs::{FileSystem, FileOps, DirEntry, Stat, FileType};

struct BiosParameterBlock {
    bytes_per_sector: u16,

```

```

    sectors_per_cluster: u8,
    reserved_sectors: u16,
    num_fats: u8,
    total_sectors_32: u32,
    fat_size_32: u32,
    root_cluster: u32,
}

impl BiosParameterBlock {
    fn parse(sector: &[u8; 512]) -> Option<Self> {
        if sector[510] != 0x55 || sector[511] != 0xAA {
            return None;
        }

        let bytes_per_sector = u16::from_le_bytes([sector[11], sector[12]]);
        let sectors_per_cluster = sector[13];
        let reserved_sectors = u16::from_le_bytes([sector[14], sector[15]]);
        let num_fats = sector[16];
        let total_sectors_32 = u32::from_le_bytes([sector[32], sector[33], sector[34], sector[35]]);
        let fat_size_32 = u32::from_le_bytes([sector[36], sector[37], sector[38], sector[39]]);
        let root_cluster = u32::from_le_bytes([sector[44], sector[45], sector[46], sector[47]]);

        if bytes_per_sector != 512 || sectors_per_cluster == 0 {
            return None;
        }

        Some(Self {
            bytes_per_sector,
            sectors_per_cluster,
            reserved_sectors,
            num_fats,
            total_sectors_32,
            fat_size_32,
            root_cluster,
        })
    }

    fn first_data_sector(&self) -> u32 {
        self.reserved_sectors as u32 + self.num_fats as u32 * self.fat_size_32
    }

    fn cluster_to_sector(&self, cluster: u32) -> u64 {
        let first_data = self.first_data_sector();
        (first_data + (cluster - 2) * self.sectors_per_cluster as u32) as u64
    }

    fn fat_sector_for_cluster(&self, cluster: u32) -> (u64, usize) {
        let fat_offset = cluster * 4;
        let fat_sector = self.reserved_sectors as u64 + fat_offset as u64 / 512;
        let offset_in_sector = (fat_offset % 512) as usize;
        (fat_sector, offset_in_sector)
    }
}

pub struct Fat32Fs {
    block: SpinLock<VirtioBlock>,
    bpb: BiosParameterBlock,
}

impl Fat32Fs {
    pub fn new(mut block: VirtioBlock) -> Option<Self> {
        let mut sector = [0u8; 512];
        if !block.read_sector(0, &mut sector) {
            crate::println!("[fat32] Failed to read boot sector");
            return None;
        }
    }
}

```

```

    }

    let bpb = BiosParameterBlock::parse(&sector)?;

    crate::println!(
        "[fat32] Mounted: {} bytes/sector, {} sectors/cluster, root cluster {}",
        bpb.bytes_per_sector,
        bpb.sectors_per_cluster,
        bpb.root_cluster
    );

    Some(Self {
        block: SpinLock::new(block),
        bpb,
    })
}

fn read_cluster(&self, cluster: u32, buf: &mut Vec<u8>)-> bool {
    let start_sector = self.bpb.cluster_to_sector(cluster);
    let count = self.bpb.sectors_per_cluster as u64;
    let mut block = self.block.lock();

    for i in 0..count {
        let mut sector = [0u8; 512];
        if !block.read_sector(start_sector + i, &mut sector) {
            return false;
        }
        buf.extend_from_slice(&sector);
    }
    true
}

fn next_cluster(&self, cluster: u32)-> Option<u32> {
    let (fat_sector, offset) = self.bpb.fat_sector_for_cluster(cluster);
    let mut sector = [0u8; 512];
    let mut block = self.block.lock();
    if !block.read_sector(fat_sector, &mut sector) {
        return None;
    }
    let entry = u32::from_le_bytes([
        sector[offset],
        sector[offset + 1],
        sector[offset + 2],
        sector[offset + 3],
    ]) & 0xFFFF_FFFF;

    if entry >= 0xFFFF_FFF8 {
        None // End of chain
    } else {
        Some(entry)
    }
}

fn read_chain(&self, start_cluster: u32)-> Vec<u8> {
    let mut data = Vec::new();
    let mut cluster = start_cluster;
    loop {
        if !self.read_cluster(cluster, &mut data) {
            break;
        }
        match self.next_cluster(cluster) {
            Some(next) => cluster = next,
            None => break,
        }
    }
}

```

```

    data
}

fn parse_dir_entries(&self, data: &[u8]) -> Vec<Fat32DirEntry> {
    let mut entries = Vec::new();
    let mut lfn_parts: Vec<(u8, [u16; 13])> = Vec::new();

    for chunk in data.chunks_exact(32) {
        if chunk[0] == 0x00 {
            break; // No more entries
        }
        if chunk[0] == 0xE5 {
            lfn_parts.clear();
            continue; // Deleted
        }

        // LFN entry
        if chunk[11] == 0x0F {
            let seq = chunk[0];
            let mut name_chars = [0u16; 13];
            // chars 1-5 at offset 1
            for i in 0..5 {
                name_chars[i] = u16::from_le_bytes([chunk[1 + i * 2], chunk[2 + i * 2]]);
            }
            // chars 6-11 at offset 14
            for i in 0..6 {
                name_chars[5 + i] = u16::from_le_bytes([chunk[14 + i * 2], chunk[15 + i * 2]]);
            }
            // chars 12-13 at offset 28
            for i in 0..2 {
                name_chars[11 + i] = u16::from_le_bytes([chunk[28 + i * 2], chunk[29 + i * 2]]);
            }
            lfn_parts.push((seq & 0x3F, name_chars));
            continue;
        }

        let attr = chunk[11];
        if attr & 0x08 != 0 {
            lfn_parts.clear();
            continue; // Volume label
        }

        let name = if !lfn_parts.is_empty() {
            lfn_parts.sort_by_key(|(seq, _)| *seq);
            let mut full_name = String::new();
            for (_, chars) in &lfn_parts {
                for &c in chars {
                    if c == 0x0000 || c == 0xFFFF {
                        break;
                    }
                    if let Some(ch) = char::from_u32(c as u32) {
                        full_name.push(ch);
                    }
                }
            }
            lfn_parts.clear();
            full_name
        } else {
            parse_8_3_name(chunk)
        };

        let file_type = if attr & 0x10 != 0 {
            FileType::Directory
        } else {
            FileType::File

```

```

    };

    let cluster_hi = u16::from_le_bytes([chunk[20], chunk[21]]) as u32;
    let cluster_lo = u16::from_le_bytes([chunk[26], chunk[27]]) as u32;
    let cluster = (cluster_hi <<< 16) | cluster_lo;
    let size = u32::from_le_bytes([chunk[28], chunk[29], chunk[30], chunk[31]]) as usize;

    if name == "." || name == ".." {
        continue;
    }

    entries.push(Fat32DirEntry {
        name,
        file_type,
        cluster,
        size,
    });
}

entries
}

fn find_entry(&self, path: &str) -> Option<Fat32DirEntry> {
    let path = path.trim_matches('/');
    if path.is_empty() {
        return Some(Fat32DirEntry {
            name: String::from("/"),
            file_type: FileType::Directory,
            cluster: self.bpb.root_cluster,
            size: 0,
        });
    }

    let parts: Vec<&str> = path.split('/').filter(|s| !s.is_empty()).collect();
    let mut current_cluster = self.bpb.root_cluster;

    for (i, part) in parts.iter().enumerate() {
        let dir_data = self.read_chain(current_cluster);
        let entries = self.parse_dir_entries(&dir_data);

        let found = entries.iter().find(|e| {
            e.name.eq_ignore_ascii_case(part)
        });

        match found {
            Some(entry) => {
                if i == parts.len() - 1 {
                    return Some(entry.clone());
                }
                if entry.file_type != FileType::Directory {
                    return None;
                }
                current_cluster = entry.cluster;
            }
            None => return None,
        }
    }

    None
}

#[derive(Clone)]
struct Fat32DirEntry {
    name: String,

```

```

        file_type: FileType,
        cluster: u32,
        size: usize,
    }

fn parse_8_3_name(entry: &[u8]) -> String {
    let mut name = String::new();
    let base = &entry[0..8];
    let ext = &entry[8..11];

    for &b in base {
        if b == 0x20 {
            break;
        }
        name.push(if b >= b'A' && b <= b'Z' {
            (b + 32) as char
        } else {
            b as char
        });
    }

    let ext_str: String = ext
        .iter()
        .take_while(|&b| b != 0x20)
        .map(|&b| {
            if b >= b'A' && b <= b'Z' {
                (b + 32) as char
            } else {
                b as char
            }
        })
        .collect();

    if !ext_str.is_empty() {
        name.push('.');
        name.push_str(&ext_str);
    }

    name
}

impl FileSystem for Fat32Fs {
    fn name(&self) -> &str {
        "fat32"
    }

    fn open(&self, path: &str) -> Result<Box<dyn FileOps>, &'static str>; {
        let entry = self.find_entry(path).ok_or("file not found");
        if entry.file_type == FileType::Directory {
            return Err("cannot open directory as file");
        }
        let data = self.read_chain(entry.cluster);
        let data = data[..entry.size].to_vec();
        Ok(Box::new(Fat32File { data, size: entry.size }))
    }

    fn readdir(&self, path: &str) -> Result<Vec<DirEntry>, &'static str>; {
        let entry = self.find_entry(path).ok_or("directory not found");
        if entry.file_type != FileType::Directory {
            return Err("not a directory");
        }
        let dir_data = self.read_chain(entry.cluster);
        let fat_entries = self.parse_dir_entries(&dir_data);
        Ok(fat_entries
            .into_iter()

```



```

        .map(|e| DirEntry {
            name: e.name,
            file_type: e.file_type,
            size: e.size,
        })
        .collect()
    }

    fn stat(&self, path: &str) -> Result<Stat, &'static str> {
        let entry = self.find_entry(path).ok_or("not found")?;
        Ok(Stat {
            file_type: entry.file_type,
            size: entry.size,
        })
    }
}

struct Fat32File {
    data: Vec<u8>,
    size: usize,
}

impl FileOps for Fat32File {
    fn read(&mut self, offset: usize, buf: &mut [u8]) -> Result<usize, &'static str> {
        if offset >= self.size {
            return Ok(0);
        }
        let available = self.size - offset;
        let to_read = buf.len().min(available);
        buf[..to_read].copy_from_slice(&self.data[offset..offset + to_read]);
        Ok(to_read)
    }

    fn write(&mut self, _offset: usize, _data: &[u8]) -> Result<usize, &'static str> {
        Err("fat32 is read-only")
    }

    fn stat(&self) -> Result<Stat, &'static str> {
        Ok(Stat {
            file_type: FileType::File,
            size: self.size,
        })
    }
}

```

9.3 RAM Filesystem

RamFS stores files and directories as an in-memory tree using BTreeMap. It supports create, read, write, and delete operations, and is mounted at /tmp.

kernel/src/fs/ramfs.rs

```

extern crate alloc;

use alloc::boxed::Box;
use alloc::collections::BTreeMap;
use alloc::string::String;
use alloc::vec::Vec;
use crate::sync::spinlock::SpinLock;
use super::vfs::{FileSystem, FileOps, DirEntry, Stat, FileType};

enum Node {
    File(Vec<u8>),

```

```

    Directory(BTreeMap<String, Node>),
}

pub struct RamFs {
    root: SpinLock<BTreeMap<String, Node>>,
}

impl RamFs {
    pub fn new() -> Self {
        Self {
            root: SpinLock::new(BTreeMap::new()),
        }
    }

    fn with_parent<F, R>(tree: &mut BTreeMap<String, Node>, path: &str, f: F) -> Result<R, &str>
    where
        F: FnOnce(&mut BTreeMap<String, Node>, &str) -> Result<R, &str> {
        {
            let path = path.trim_matches('/');
            if path.is_empty() {
                return Err("invalid path");
            }

            let parts: Vec<&str> = path.split('/').collect();
            if parts.len() == 1 {
                return f(tree, parts[0]);
            }

            let mut current = tree;
            for &part in &parts[..parts.len() - 1] {
                let node = current.get_mut(part).ok_or("parent directory not found");
                match node {
                    Node::Directory(ref mut children) => current = children,
                    _ => return Err("not a directory"),
                }
            }
            f(current, parts[parts.len() - 1])
        }
    }

    fn lookup<A>(tree: &BTreeMap<String, Node>, path: &str) -> Option<&A Node> {
        let path = path.trim_matches('/');
        if path.is_empty() {
            return None; // Root is special
        }

        let parts: Vec<&str> = path.split('/').collect();
        let mut current = tree;
        for (i, &part) in parts.iter().enumerate() {
            match current.get(part) {
                Some(node) => {
                    if i == parts.len() - 1 {
                        return Some(node);
                    }
                    match node {
                        Node::Directory(ref children) => current = children,
                        _ => return None,
                    }
                }
                None => return None,
            }
        }
        None
    }

    fn dir_entries(tree: &BTreeMap<String, Node>) -> Vec<DirEntry> {

```

```

        tree.iter()
            .map(|(name, node)| {
                let (file_type, size) = match node {
                    Node::File(data) => (FileType::File, data.len()),
                    Node::Directory(_) => (FileType::Directory, 0),
                };
                DirEntry {
                    name: name.clone(),
                    file_type,
                    size,
                }
            })
            .collect()
    }
}

impl FileSystem for RamFs {
    fn name(&self) -> &str {
        "ramfs"
    }

    fn open(&self, path: &str) -> Result<Box<dyn FileOps>, &'static str> {
        let path = path.trim_matches('/');
        if path.is_empty() {
            return Err("cannot open root as file");
        }
        let tree = self.root.lock();
        match Self::lookup(&tree, path) {
            Some(Node::File(data)) => Ok(Box::new(RamFile {
                path: String::from(path),
                data: data.clone(),
            })),
            Some(Node::Directory(_)) => Err("cannot open directory as file"),
            None => Err("file not found"),
        }
    }

    fn readdir(&self, path: &str) -> Result<Vec<DirEntry>, &'static str> {
        let path = path.trim_matches('/');
        let tree = self.root.lock();
        if path.is_empty() {
            return Ok(Self::dir_entries(&tree));
        }
        match Self::lookup(&tree, path) {
            Some(Node::Directory(children)) => Ok(Self::dir_entries(children)),
            Some(Node::File(_)) => Err("not a directory"),
            None => Err("directory not found"),
        }
    }

    fn stat(&self, path: &str) -> Result<Stat, &'static str> {
        let path = path.trim_matches('/');
        if path.is_empty() {
            return Ok(Stat {
                file_type: FileType::Directory,
                size: 0,
            });
        }
        let tree = self.root.lock();
        match Self::lookup(&tree, path) {
            Some(Node::File(data)) => Ok(Stat {
                file_type: FileType::File,
                size: data.len(),
            }),
            Some(Node::Directory(_)) => Ok(Stat {

```

```

        file_type: FileType::Directory,
        size: 0,
    }},
    None => Err("not found"),
}

}

fn create(&self, path: &str, file_type: FileType) -> Result<(), &'static str> {
    let mut tree = self.root.lock();
    Self::with_parent(&mut tree, path, |parent, name| {
        if parent.contains_key(name) {
            return Err("already exists");
        }
        let node = match file_type {
            FileType::File => Node::File(Vec::new()),
            FileType::Directory => Node::Directory(BTreeMap::new()),
        };
        parent.insert(String::from(name), node);
        Ok(())
    })
}

fn remove(&self, path: &str) -> Result<(), &'static str> {
    let mut tree = self.root.lock();
    Self::with_parent(&mut tree, path, |parent, name| {
        parent.remove(name).ok_or("not found")?;
        Ok(())
    })
}

}

struct RamFile {
    path: String,
    data: Vec<u8>,
}

impl FileOps for RamFile {
    fn read(&mut self, offset: usize, buf: &mut [u8]) -> Result<usize, &'static str> {
        if offset >= self.data.len() {
            return Ok(0);
        }
        let available = self.data.len() - offset;
        let to_read = buf.len().min(available);
        buf[..to_read].copy_from_slice(&self.data[offset..offset + to_read]);
        Ok(to_read)
    }

    fn write(&mut self, offset: usize, data: &[u8]) -> Result<usize, &'static str> {
        let end = offset + data.len();
        if end > self.data.len() {
            self.data.resize(end, 0);
        }
        self.data[offset..end].copy_from_slice(data);
        Ok(data.len())
    }

    fn stat(&self) -> Result<Stat, &'static str> {
        Ok(Stat {
            file_type: FileType::File,
            size: self.data.len(),
        })
    }

    fn close(&mut self) {
        // RamFile writes are in-memory only in this clone.
    }
}

```

```

        // For persistence, we'd need to write back to the RamFs tree.
        // This is handled via a write-back on close.
    }
}

pub struct RamFileWriteback<&lt;'a>> {
    fs: &&'a RamFs,
    path: String,
    data: Vec<&lt;u8>>,
}

impl RamFs {
    pub fn write_file(&&self, path: &&str, data: &&[u8]) -> Result<&lt;(), &&'static str>> {
        let mut tree = self.root.lock();
        Self::with_parent(&&mut tree, path, |parent, name| {
            match parent.get_mut(name) {
                Some(Node::File(ref mut existing)) => {
                    existing.clear();
                    existing.extend_from_slice(data);
                    Ok(())
                }
                Some(Node::Directory(_)) => Err("is a directory"),
                None => {
                    parent.insert(String::from(name), Node::File(data.to_vec()));
                    Ok(())
                }
            }
        })
    }

    pub fn read_file(&&self, path: &&str) -> Result<&lt;Vec<&lt;u8>>, &&'static str>> {
        let tree = self.root.lock();
        match Self::lookup(&&tree, path) {
            Some(Node::File(data)) => Ok(data.clone()),
            Some(Node::Directory(_)) => Err("is a directory"),
            None => Err("file not found"),
        }
    }
}

```

10. Phase 8: Network Stack

The network stack implements a layered architecture from Ethernet frames up to TCP connections. The guest IP is 10.0.2.15 (QEMU user-mode networking default), with gateway at 10.0.2.2 and subnet mask 255.255.255.0.

10.1 Ethernet Frame Layer

Handles frame parsing and construction. Dispatches by EtherType: 0x0800 for IPv4, 0x0806 for ARP.

kernel/src/net/ethernet.rs

```
use alloc::vec::Vec;

pub const ETHERTYPE_IPV4: u16 = 0x0800;
pub const ETHERTYPE_ARP: u16 = 0x0806;

pub struct EthernetFrame<'a> {
    pub dst_mac: [u8; 6],
    pub src_mac: [u8; 6],
    pub ethertype: u16,
    pub payload: &'a [u8],
}

impl<'a> EthernetFrame<'a> {
    pub fn parse(data: &'a [u8]) -> Option<Self> {
        if data.len() < 14 {
            return None;
        }
        let mut dst_mac = [0u8; 6];
        let mut src_mac = [0u8; 6];
        dst_mac.copy_from_slice(&data[0..6]);
        src_mac.copy_from_slice(&data[6..12]);
        let ethertype = u16::from_be_bytes([data[12], data[13]]);
        Some(Self {
            dst_mac,
            src_mac,
            ethertype,
            payload: &data[14..],
        })
    }
}

pub fn build_frame(dst_mac: &[u8; 6], ethertype: u16, payload: &[u8]) -> Vec<u8> {
    let src_mac = super::our_mac();
    let mut frame = Vec::with_capacity(14 + payload.len());
    frame.extend_from_slice(dst_mac);
    frame.extend_from_slice(&src_mac);
    frame.extend_from_slice(&ethertype.to_be_bytes());
    frame.extend_from_slice(payload);
    frame
}

pub fn handle_frame(data: &[u8]) {
    let frame = match EthernetFrame::parse(data) {
        Some(f) => f,
        None => return,
    };
};
```

```

    match frame.ethertype {
        ETHERTYPE_ARP => super::arp::handle_arp(frame.payload),
        ETHERTYPE_IPV4 => super::ipv4::handle_ipv4(frame.payload, &frame.src_mac),
        _ => {}
    }
}

```

10.2 ARP (Address Resolution Protocol)

Maintains a BTreeMap-based ARP table. Responds to ARP requests for our IP and resolves MAC addresses for outgoing packets. Resolution includes a polling loop that processes incoming frames while waiting for the reply.

kernel/src/net/arp.rs

```

extern crate alloc;

use alloc::collections::BTreeMap;
use crate::sync::spinlock::SpinLock;
use super::ethernet;

static ARP_TABLE: SpinLock<Option<BTreeMap<[u8; 4], [u8; 6]>>> = SpinLock::new(None);

const ARP_REQUEST: u16 = 1;
const ARP_REPLY: u16 = 2;

pub fn init() {
    *ARP_TABLE.lock() = Some(BTreeMap::new());
}

pub fn lookup(ip: &[u8; 4]) -> Option<[u8; 6]> {
    let table = ARP_TABLE.lock();
    table.as_ref().and_then(|t| t.get(ip).copied())
}

pub fn resolve(ip: &[u8; 4]) -> Option<[u8; 6]> {
    if let Some(mac) = lookup(ip) {
        return Some(mac);
    }

    send_request(ip);

    for _ in 0..100_000 {
        super::poll();
        if let Some(mac) = lookup(ip) {
            return Some(mac);
        }
        core::hint::spin_loop();
    }

    None
}

fn insert(ip: [u8; 4], mac: [u8; 6]) {
    let mut table = ARP_TABLE.lock();
    if let Some(ref mut t) = *table {
        t.insert(ip, mac);
    }
}

pub fn handle_arp(data: &[u8]) {
    if data.len() < 28 {
        return;
    }
}

```

```

    }

    let htype = u16::from_be_bytes([data[0], data[1]]);
    let ptype = u16::from_be_bytes([data[2], data[3]]);
    let hlen = data[4];
    let plen = data[5];
    let oper = u16::from_be_bytes([data[6], data[7]]);

    if htype != 1 || ptype != 0x0800 || hlen != 6 || plen != 4 {
        return;
    }

    let mut sender_mac = [0u8; 6];
    let mut sender_ip = [0u8; 4];
    let mut target_ip = [0u8; 4];

    sender_mac.copy_from_slice(&data[8..14]);
    sender_ip.copy_from_slice(&data[14..18]);
    target_ip.copy_from_slice(&data[24..28]);

    insert(sender_ip, sender_mac);

    if oper == ARP_REQUEST && target_ip == super::OUR_IP {
        send_reply(&sender_mac, &sender_ip);
    }
}

fn send_request(target_ip: &[u8; 4]) {
    let our_mac = super::our_mac();
    let mut arp = [0u8; 28];
    arp[0..2].copy_from_slice(&u16::to_be_bytes()); // htype: Ethernet
    arp[2..4].copy_from_slice(&0x0800u16::to_be_bytes()); // ptype: IPv4
    arp[4] = 6; // hlen
    arp[5] = 4; // plen
    arp[6..8].copy_from_slice(&ARP_REQUEST.to_be_bytes());
    arp[8..14].copy_from_slice(&our_mac);
    arp[14..18].copy_from_slice(&super::OUR_IP);
    arp[18..24].copy_from_slice(&[0xFF; 6]);
    arp[24..28].copy_from_slice(target_ip);

    let broadcast = [0xFF; 6];
    let frame = ethernet::build_frame(&broadcast, ethernet::ETHERTYPE_ARP, &arp);
    super::send_frame(&frame);
}

fn send_reply(target_mac: &[u8; 6], target_ip: &[u8; 4]) {
    let our_mac = super::our_mac();
    let mut arp = [0u8; 28];
    arp[0..2].copy_from_slice(&u16::to_be_bytes());
    arp[2..4].copy_from_slice(&0x0800u16::to_be_bytes());
    arp[4] = 6;
    arp[5] = 4;
    arp[6..8].copy_from_slice(&ARP_REPLY.to_be_bytes());
    arp[8..14].copy_from_slice(&our_mac);
    arp[14..18].copy_from_slice(&super::OUR_IP);
    arp[18..24].copy_from_slice(target_mac);
    arp[24..28].copy_from_slice(target_ip);

    let frame = ethernet::build_frame(target_mac, ethernet::ETHERTYPE_ARP, &arp);
    super::send_frame(&frame);
}

```

10.3 IPv4

Parses and constructs IPv4 headers with internet checksum calculation. Handles TTL=64, Don't Fragment flag, and subnet-based next-hop routing (same subnet = direct, otherwise = gateway).

kernel/src/net/ipv4.rs

```
use alloc::vec::Vec;
use super::ethernet;

pub struct Ipv4Packet<'a> {
    pub src_ip: [u8; 4],
    pub dst_ip: [u8; 4],
    pub protocol: u8,
    pub payload: &'a [u8],
    pub header: &'a [u8],
}

pub const PROTO_ICMP: u8 = 1;
pub const PROTO_TCP: u8 = 6;
pub const PROTO_UDP: u8 = 17;

impl<'a> Ipv4Packet<'a> {
    pub fn parse(data: &'a [u8]) -> Option<Self> {
        if data.len() < 20 {
            return None;
        }

        let version = data[0] >> 4;
        if version != 4 {
            return None;
        }

        let ihl = (data[0] & 0xF) as usize * 4;
        if ihl < 20 || data.len() < ihl {
            return None;
        }

        let total_len = u16::from_be_bytes([data[2], data[3]]) as usize;
        if data.len() < total_len {
            return None;
        }

        let protocol = data[9];
        let mut src_ip = [0u8; 4];
        let mut dst_ip = [0u8; 4];
        src_ip.copy_from_slice(&data[12..16]);
        dst_ip.copy_from_slice(&data[16..20]);

        Some(Self {
            src_ip,
            dst_ip,
            protocol,
            payload: &data[ihl..total_len],
            header: &data[..ihl],
        })
    }
}

pub fn checksum(data: &'a [u8]) -> u16 {
    let mut sum: u32 = 0;
    let mut i = 0;
    while i + 1 < data.len() {
        sum += u16::from_be_bytes([data[i], data[i + 1]]) as u32;
        i += 2;
    }
    if i < data.len() {

```

```

        sum += (data[i] as u32) <<< 8;
    }
    while sum >>> 16 != 0 {
        sum = (sum & 0xFFFF) + (sum >>> 16);
    }
    !(sum as u16)
}

pub fn send_packet(dst_ip: &[u8; 4], protocol: u8, payload: &[u8]) -> bool {
    let total_len = 20 + payload.len();
    let mut packet = Vec::with_capacity(total_len);

    // IPv4 header
    packet.push(0x45); // version=4, IHL=5
    packet.push(0x00); // DSCP/ECN
    packet.extend_from_slice(&(total_len as u16).to_be_bytes());
    packet.extend_from_slice(&[0x00, 0x00]); // identification
    packet.extend_from_slice(&[0x40, 0x00]); // flags=DF, fragment offset=0
    packet.push(64); // TTL
    packet.push(protocol);
    packet.extend_from_slice(&[0x00, 0x00]); // checksum placeholder
    packet.extend_from_slice(&super::OUR_IP);
    packet.extend_from_slice(dst_ip);

    // Calculate header checksum
    let cksum = checksum(&packet[..20]);
    packet[10] = (cksum >>> 8) as u8;
    packet[11] = (cksum & 0xFF) as u8;

    packet.extend_from_slice(payload);

    let next_hop = if same_subnet(dst_ip) { *dst_ip } else { super::GATEWAY_IP };
    let dst_mac = match super::arp::resolve(&next_hop) {
        Some(mac) => mac,
        None => {
            crate::println!("[ipv4] ARP resolution failed for {}.{}.{}.{}",
                next_hop[0], next_hop[1], next_hop[2], next_hop[3]);
            return false;
        }
    };

    let frame = ethernet::build_frame(&dst_mac, ethernet::ETHERTYPE_IPV4, &packet);
    super::send_frame(&frame)
}

fn same_subnet(ip: &[u8; 4]) -> bool {
    for i in 0..4 {
        if (ip[i] & super::SUBNET_MASK[i]) != (super::OUR_IP[i] & super::SUBNET_MASK[i]) {
            return false;
        }
    }
    true
}

pub fn handle_ipv4(data: &[u8], _src_mac: &[u8; 6]) {
    let packet = match Ipv4Packet::parse(data) {
        Some(p) => p,
        None => return,
    };

    if packet.dst_ip != super::OUR_IP {
        return;
    }

    match packet.protocol {

```

```

        PROTO_ICMP => super::icmp::handle_icmp(&packet),
        PROTO_UDP => super::udp::handle_udp(&packet),
        PROTO_TCP => super::tcp::handle_tcp(&packet),
        _ => {}
    }
}

```

10.4 ICMP

Responds to Echo Request with Echo Reply, enabling ping from the host.

kernel/src/net/icmp.rs

```

use super::ipv4;

const ICMP_ECHO_REQUEST: u8 = 8;
const ICMP_ECHO_REPLY: u8 = 0;

pub fn handle_icmp(packet: &ipv4::Ipv4Packet) {
    if packet.payload.len() < 8 {
        return;
    }

    let icmp_type = packet.payload[0];
    let icmp_code = packet.payload[1];

    if icmp_type == ICMP_ECHO_REQUEST && icmp_code == 0 {
        send_echo_reply(packet);
    }
}

fn send_echo_reply(request: &ipv4::Ipv4Packet) {
    let mut reply = alloc::vec![0u8; request.payload.len()];
    reply.copy_from_slice(request.payload);

    reply[0] = ICMP_ECHO_REPLY;
    reply[1] = 0;
    // Zero checksum before recalculating
    reply[2] = 0;
    reply[3] = 0;

    let cksum = ipv4::checksum(&reply);
    reply[2] = (cksum >> 8) as u8;
    reply[3] = (cksum & 0xFF) as u8;

    ipv4::send_packet(&request.src_ip, ipv4::PROTO_ICMP, &reply);
}

```

10.5 UDP

Stateless UDP with a port dispatch table. Applications bind handlers to ports. A UDP echo server runs on port 7.

kernel/src/net/udp.rs

```

extern crate alloc;

use alloc::collections::BTreeMap;
use alloc::vec::Vec;
use crate::sync::spinlock::SpinLock;
use super::ipv4;

```

```

type UdpHandler = fn(src_ip: [u8; 4], src_port: u16, data: &[u8]);

static UDP_PORTS: SpinLock<&lt;Option<&lt;BTreeMap<&lt;u16, UdpHandler>>>>> = SpinLock::new(None);

pub fn init() {
    *UDP_PORTS.lock() = Some(BTreeMap::new());
}

pub fn bind(port: u16, handler: UdpHandler) {
    let mut ports = UDP_PORTS.lock();
    if let Some(ref mut map) = *ports {
        map.insert(port, handler);
    }
}

pub fn send(dst_ip: &[u8; 4], dst_port: u16, src_port: u16, data: &[u8]) -> bool {
    let udp_len = 8 + data.len();
    let mut packet = Vec::with_capacity(udp_len);

    packet.extend_from_slice(&src_port.to_be_bytes());
    packet.extend_from_slice(&dst_port.to_be_bytes());
    packet.extend_from_slice(&(udp_len as u16).to_be_bytes());
    packet.extend_from_slice(&[0x00, 0x00]); // checksum (optional for UDP over IPv4)

    packet.extend_from_slice(data);

    ipv4::send_packet(dst_ip, ipv4::PROTO_UDP, &packet)
}

pub fn handle_udp(ip_packet: &ipv4::Ipv4Packet) {
    let data = ip_packet.payload;
    if data.len() < 8 {
        return;
    }

    let src_port = u16::from_be_bytes([data[0], data[1]]);
    let dst_port = u16::from_be_bytes([data[2], data[3]]);
    let payload = &data[8..];

    let handler = {
        let ports = UDP_PORTS.lock();
        ports.as_ref().and_then(|map| map.get(&dst_port).copied())
    };

    if let Some(handler) = handler {
        handler(ip_packet.src_ip, src_port, payload);
    }
}

```

10.6 TCP

Simplified TCP state machine implementing LISTEN, SYN_RECEIVED, ESTABLISHED, CLOSE_WAIT, LAST_ACK, FIN_WAIT1, FIN_WAIT2, and CLOSED states. Supports passive open (server), connection tracking with BTreeMap, proper TCP checksum with pseudo-header, and data echo for testing. A TCP echo server listens on port 8080.

kernel/src/net/tcp.rs

```

extern crate alloc;

use alloc::collections::BTreeMap;
use alloc::vec::Vec;

```

```

use crate::sync::spinlock::SpinLock;
use super::ipv4;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
enum TcpState {
    Listen,
    SynReceived,
    Established,
    FinWait1,
    FinWait2,
    CloseWait,
    LastAck,
    Closed,
}

const TH_FIN: u8 = 0x01;
const TH_SYN: u8 = 0x02;
const TH_RST: u8 = 0x04;
const TH_PSH: u8 = 0x08;
const TH_ACK: u8 = 0x10;

#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
struct TcpKey {
    remote_ip: [u8; 4],
    remote_port: u16,
    local_port: u16,
}

struct TcpConnection {
    state: TcpState,
    snd_nxt: u32,
    snd_una: u32,
    rcv_nxt: u32,
    local_port: u16,
    remote_port: u16,
    remote_ip: [u8; 4],
    rx_buf: Vec<u8>;,
}

type TcpAcceptHandler = fn(local_port: u16, remote_ip: [u8; 4], remote_port: u16);

static TCP_STATE: SpinLock<Option<TcpGlobal>> = SpinLock::new(None);

struct TcpGlobal {
    connections: BTreeMap<TcpKey, TcpConnection>;,
    listeners: BTreeMap<u16, TcpAcceptHandler>;,
    isn_counter: u32,
}

pub fn init() {
    *TCP_STATE.lock() = Some(TcpGlobal {
        connections: BTreeMap::new(),
        listeners: BTreeMap::new(),
        isn_counter: 1000,
    });
}

pub fn listen(port: u16, handler: TcpAcceptHandler) {
    let mut state = TCP_STATE.lock();
    if let Some(ref mut s) = *state {
        s.listeners.insert(port, handler);
        crate::println!("[tcp] Listening on port {}", port);
    }
}

```

```

pub fn handle_tcp(ip_packet: &ipv4::Ipv4Packet) {
    let data = ip_packet.payload;
    if data.len() < 20 {
        return;
    }

    let src_port = u16::from_be_bytes([data[0], data[1]]);
    let dst_port = u16::from_be_bytes([data[2], data[3]]);
    let seq_num = u32::from_be_bytes([data[4], data[5], data[6], data[7]]);
    let ack_num = u32::from_be_bytes([data[8], data[9], data[10], data[11]]);
    let data_offset = ((data[12] >> 4) as usize) * 4;
    let flags = data[13];

    if data_offset > data.len() {
        return;
    }

    let payload = &data[data_offset..];
    let key = TcpKey {
        remote_ip: ip_packet.src_ip,
        remote_port: src_port,
        local_port: dst_port,
    };

    let mut state = TCP_STATE.lock();
    let s = match state.as_mut() {
        Some(s) => s,
        None => return,
    };

    if let Some(conn) = s.connections.get_mut(&key) {
        match conn.state {
            TcpState::SynReceived => {
                if flags & TH_ACK != 0 {
                    conn.state = TcpState::Established;
                    conn.snd_una = ack_num;
                    crate::println!("[tcp] Connection established {}:{}",
                        format_ip(&ip_packet.src_ip), src_port);
                }
            }
            TcpState::Established => {
                if flags & TH_FIN != 0 {
                    conn.rcv_next = seq_num.wrapping_add(1);
                    conn.state = TcpState::CloseWait;
                    send_tcp_packet(conn, TH_ACK, &[]);
                    conn.state = TcpState::LastAck;
                    send_tcp_packet(conn, TH_FIN | TH_ACK, &[]);
                    return;
                }

                if !payload.is_empty() {
                    conn.rcv_next = seq_num.wrapping_add(payload.len() as u32);
                    conn.rx_buf.extend_from_slice(payload);
                    send_tcp_packet(conn, TH_ACK, &[]);

                    crate::println!("[tcp] Received {} bytes on port {}", payload.len(), dst_port);

                    // Echo back for testing
                    let echo = conn.rx_buf.clone();
                    conn.rx_buf.clear();
                    send_tcp_packet(conn, TH_ACK | TH_PSH, &echo);
                } else if flags & TH_ACK != 0 {
                    conn.snd_una = ack_num;
                }
            }
        }
    }
}

```

```

        TcpState::LastAck => {
            if flags & TH_ACK != 0 {
                conn.state = TcpState::Closed;
            }
        }
        TcpState::FinWait1 => {
            if flags & TH_ACK != 0 {
                conn.state = TcpState::FinWait2;
            }
        }
        TcpState::FinWait2 => {
            if flags & TH_FIN != 0 {
                conn.rcv_nxt = seq_num.wrapping_add(1);
                send_tcp_packet(conn, TH_ACK, &[]);
                conn.state = TcpState::Closed;
            }
        }
        _ => {}
    }
    return;
}

// No existing connection – check listeners for SYN
if flags & TH_SYN != 0 && flags & TH_ACK == 0 {
    if let Some(&handler) = s.listeners.get(&dst_port) {
        let isn = s.isn_counter;
        s.isn_counter = s.isn_counter.wrapping_add(64000);

        let mut conn = TcpConnection {
            state: TcpState::SynReceived,
            snd_nxt: isn.wrapping_add(1),
            snd_una: isn,
            rcv_nxt: seq_num.wrapping_add(1),
            local_port: dst_port,
            remote_port: src_port,
            remote_ip: ip_packet.src_ip,
            rx_buf: Vec::new(),
        };

        send_tcp_packet(&mut conn, TH_SYN | TH_ACK, &[]);
        s.connections.insert(key, conn);
        handler(dst_port, ip_packet.src_ip, src_port);
    } else {
        // Send RST for unlisted port
        send_rst(&ip_packet.src_ip, src_port, dst_port, ack_num, seq_num.wrapping_add(1));
    }
}

fn send_tcp_packet(conn: &mut TcpConnection, flags: u8, payload: &[u8]) {
    let header_len = 20u8;
    let total_len = header_len as usize + payload.len();
    let mut segment = Vec::with_capacity(total_len);

    segment.extend_from_slice(&conn.local_port.to_be_bytes());
    segment.extend_from_slice(&conn.remote_port.to_be_bytes());
    segment.extend_from_slice(&conn.snd_nxt.to_be_bytes());
    segment.extend_from_slice(&conn.rcv_nxt.to_be_bytes());
    segment.push((header_len / 4) << 4); // data offset
    segment.push(flags);
    segment.extend_from_slice(&8192u16.to_be_bytes()); // window
    segment.extend_from_slice(&[0, 0]); // checksum placeholder
    segment.extend_from_slice(&[0, 0]); // urgent pointer
    segment.extend_from_slice(payload);
}

```

```

// TCP checksum with pseudo-header
let cksum = tcp_checksum(&super::OUR_IP, &conn.remote_ip, &segment);
segment[16] = (cksum >> 8) as u8;
segment[17] = (cksum & 0xFF) as u8;

if flags & TH_SYN != 0 || flags & TH_FIN != 0 {
    conn.snd_nxt = conn.snd_nxt.wrapping_add(1);
}
conn.snd_nxt = conn.snd_nxt.wrapping_add(payload.len() as u32);

ipv4::send_packet(&conn.remote_ip, ipv4::PROTO_TCP, &segment);
}

fn send_rst(dst_ip: &[u8; 4], dst_port: u16, src_port: u16, seq: u32, ack: u32) {
    let mut segment = Vec::with_capacity(20);
    segment.extend_from_slice(&src_port.to_be_bytes());
    segment.extend_from_slice(&dst_port.to_be_bytes());
    segment.extend_from_slice(&seq.to_be_bytes());
    segment.extend_from_slice(&ack.to_be_bytes());
    segment.push(5 << 4); // data offset
    segment.push(TH_RST | TH_ACK);
    segment.extend_from_slice(&0u16.to_be_bytes()); // window
    segment.extend_from_slice(&[0, 0]); // checksum
    segment.extend_from_slice(&[0, 0]); // urgent

    let cksum = tcp_checksum(&super::OUR_IP, dst_ip, &segment);
    segment[16] = (cksum >> 8) as u8;
    segment[17] = (cksum & 0xFF) as u8;

    ipv4::send_packet(dst_ip, ipv4::PROTO_TCP, &segment);
}

fn tcp_checksum(src_ip: &[u8; 4], dst_ip: &[u8; 4], segment: &[u8]) -> u16 {
    let mut pseudo = Vec::with_capacity(12 + segment.len());
    pseudo.extend_from_slice(src_ip);
    pseudo.extend_from_slice(dst_ip);
    pseudo.push(0);
    pseudo.push(ipv4::PROTO_TCP);
    pseudo.extend_from_slice(&(segment.len() as u16).to_be_bytes());
    pseudo.extend_from_slice(segment);
    ipv4::checksum(&pseudo)
}

fn format_ip(ip: &[u8; 4]) -> alloc::string::String {
    alloc::format!("{}", ip[0], ip[1], ip[2], ip[3])
}

```

11. Phase 9: System Call Interface

System calls use the AArch64 SVC (Supervisor Call) instruction. The convention is: syscall number in x8, arguments in x0-x5, return value in x0. When SVC executes at EL0, the CPU takes a synchronous exception to the Lower EL AArch64 vector. The handler checks `ESR_EL1.EC == 0x15` (SVC from AArch64) and dispatches to the syscall handler, which modifies x0 in the `ExceptionContext` for the return value.

Number	Name	Arguments	Description
0	exit	x0=code	Terminate process
1	write	x0=fd, x1=buf, x2=len	Write to file descriptor
2	read	x0=fd, x1=buf, x2=len	Read from file descriptor
3	open	x0=path, x1=len	Open file
4	close	x0=fd	Close file descriptor
7	yield	(none)	Yield CPU to scheduler
8	getpid	(none)	Get current process ID
9	sleep	x0=ms	Sleep for milliseconds

kernel/src/syscall/mod.rs

```
pub mod handlers;

use crate::arch::aarch64::exceptions::ExceptionContext;

pub const SYS_EXIT: u64 = 0;
pub const SYS_WRITE: u64 = 1;
pub const SYS_READ: u64 = 2;
pub const SYS_OPEN: u64 = 3;
pub const SYS_CLOSE: u64 = 4;
pub const SYS_YIELD: u64 = 7;
pub const SYS_GETPID: u64 = 8;
pub const SYS_SLEEP: u64 = 9;

pub fn dispatch(ctx: &mut ExceptionContext) {
    let syscall_num = ctx.gpr[8]; // x8
    let a0 = ctx.gpr[0]; // x0
    let a1 = ctx.gpr[1]; // x1
    let a2 = ctx.gpr[2]; // x2

    let result = match syscall_num {
        SYS_EXIT => handlers::sys_exit(a0),
        SYS_WRITE => handlers::sys_write(a0, a1, a2),
        SYS_READ => handlers::sys_read(a0, a1, a2),
        SYS_OPEN => handlers::sys_open(a0, a1),
        SYS_CLOSE => handlers::sys_close(a0),
        SYS_YIELD => handlers::sys_yield(),
        SYS_GETPID => handlers::sys_getpid(),
```

```

        SYS_SLEEP => handlers::sys_sleep(a0),
        _ => {
            crate::println!("[syscall] Unknown syscall {}", syscall_num);
            (-1i64) as u64
        }
    };

    ctx.gpr[0] = result;
}

```

kernel/src/syscall/handlers.rs

```

pub fn sys_exit(code: u64) -> u64 {
    crate::println!("[syscall] exit({})", code);
    loop {
        core::hint::spin_loop();
    }
}

pub fn sys_write(fd: u64, buf_ptr: u64, len: u64) -> u64 {
    if fd == 1 || fd == 2 {
        // stdout / stderr -> UART
        let len = len as usize;
        let ptr = buf_ptr as *const u8;
        for i in 0..len {
            let b = unsafe { core::ptr::read(ptr.add(i)) };
            crate::print!("{}", b as char);
        }
        len as u64
    } else {
        (-1i64) as u64
    }
}

pub fn sys_read(fd: u64, _buf_ptr: u64, _len: u64) -> u64 {
    let _ = fd;
    // stdin not implemented yet
    0
}

pub fn sys_open(path_ptr: u64, path_len: u64) -> u64 {
    let _ = (path_ptr, path_len);
    // File descriptor management not implemented yet
    (-1i64) as u64
}

pub fn sys_close(fd: u64) -> u64 {
    let _ = fd;
    0
}

pub fn sys_yield() -> u64 {
    crate::sched::schedule();
    0
}

pub fn sys_getpid() -> u64 {
    // Return current task ID
    0
}

pub fn sys_sleep(ms: u64) -> u64 {
    let _ = ms;
    // Basic sleep: just yield for now
    crate::sched::schedule();
    0
}

```

}

12. Phase 10: Userspace & Init Process

12.1 ELF Loader

The init process binary is compiled as a separate Rust crate targeting aarch64-unknown-none, linked at virtual address 0x400000. The ELF binary is embedded in the kernel via `include_bytes!`. The ELF loader parses the ELF64 header, iterates `PT_LOAD` segments, allocates physical pages, copies segment data, and maps them into a freshly allocated user page table (TTBR0).

12.2 User Page Table & EL0 Transition

A critical challenge: the kernel executes from the lower-half address space (0x4008_0000), which is mapped via TTBR0. When we switch TTBR0 to the user page table, the kernel code must still be accessible for the instructions between the TTBR0 write and the `eret`. This is solved by merging the boot page table entries into the user page table at L1/L2 level, handling collisions where user and kernel share the same L0 index.

The EL0 transition sets TTBR0 to the user page table, flushes TLBs, sets `SP_EL0` to the user stack top (0x8000_0000), `ELR_EL1` to the ELF entry point, `SPSR_EL1` to 0 (EL0t with interrupts enabled), and executes `eret`.

kernel/src/sched/process.rs

```
use crate::mm::{pmm, vmm, VirtAddr, PAGE_SIZE};
use crate::mm::vmm::PageTable;

const USER_STACK_TOP: usize = 0x8000_0000;
const USER_STACK_PAGES: usize = 4; // 16 KB user stack

static INIT_ELF: &[u8] = include_bytes!("../../init.elf");

#[repr(C)]
struct Elf64Header {
    e_ident: [u8; 16],
    e_type: u16,
    e_machine: u16,
    e_version: u32,
    e_entry: u64,
    e_phoff: u64,
    e_shoff: u64,
    e_flags: u32,
    e_ehsize: u16,
    e_phentsize: u16,
    e_phnum: u16,
}

#[repr(C)]
struct Elf64Phdr {
    p_type: u32,
    p_flags: u32,
    p_offset: u64,
    p_vaddr: u64,
```

```

    p_paddr: u64,
    p_filesz: u64,
    p_memsz: u64,
    p_align: u64,
}

const PT_LOAD: u32 = 1;
const PF_X: u32 = 1;
const PF_W: u32 = 2;

pub fn spawn_init() {
    crate::println!("[process] Loading init ELF ({} bytes)", INIT_ELF.len());

    // Validate ELF magic
    if INIT_ELF.len() < 64 || &INIT_ELF[0..4] != b"\x7FELF" {
        crate::println!("[process] Invalid ELF");
        return;
    }

    let header = unsafe { &*(INIT_ELF.as_ptr() as *const Elf64Header) };

    if header.e_machine != 0xB7 {
        crate::println!("[process] Not AArch64 ELF");
        return;
    }

    let entry = header.e_entry as usize;
    crate::println!("[process] Entry point: {:#x}", entry);

    // Allocate a fresh L0 page table for user address space
    let l0_page = pmm::alloc_page().expect("OOM for user L0 page table");
    let l0 = unsafe { &mut *(l0_page.as_usize() as *mut vmm::PageTable) };

    // Load PT_LOAD segments
    let phdr_base = unsafe { INIT_ELF.as_ptr().add(header.e_phoff as usize) };
    for i in 0..header.e_phnum {
        let phdr = unsafe { &*(phdr_base.add(i as usize * header.e_phentsize as usize) as *const Elf64Phdr) };

        if phdr.p_type != PT_LOAD {
            continue;
        }

        let vaddr_start = phdr.p_vaddr as usize;
        let memsz = phdr.p_memsz as usize;
        let filesz = phdr.p_filesz as usize;
        let offset = phdr.p_offset as usize;

        let executable = phdr.p_flags & PF_X != 0;
        let writable = phdr.p_flags & PF_W != 0;

        crate::println!(
            "[process] LOAD: vaddr={:#x} memsz={:#x} filesz={:#x} {}{}",
            vaddr_start, memsz, filesz,
            if executable { "X" } else { "-" },
            if writable { "W" } else { "R" },
        );

        // Map pages for this segment
        let page_start = vaddr_start & !(PAGE_SIZE - 1);
        let page_end = (vaddr_start + memsz + PAGE_SIZE - 1) & !(PAGE_SIZE - 1);

        for va in (page_start..page_end).step_by(PAGE_SIZE) {
            let pa = pmm::alloc_page().expect("OOM mapping user segment");

            // Zero the page first

```

```

    unsafe {
        core::ptr::write_bytes(pa.as_usize() as *mut u8, 0, PAGE_SIZE);
    }

    // Copy file data if within filesz range
    let page_offset_in_seg = va.saturating_sub(vaddr_start);
    if page_offset_in_seg < filesz {
        let src_offset = offset + page_offset_in_seg;
        let copy_start = if va < vaddr_start { vaddr_start - va } else { 0 };
        let copy_len = (filesz - page_offset_in_seg).min(PAGE_SIZE - copy_start);
        if src_offset + copy_len <= INIT_ELF.len() {
            unsafe {
                core::ptr::copy_nonoverlapping(
                    INIT_ELF.as_ptr().add(src_offset),
                    (pa.as_usize() + copy_start) as *mut u8,
                    copy_len,
                );
            }
        }
    }

    let flags = vmm::MappingFlags {
        writable,
        executable,
        user: true,
        device: false,
    };
    unsafe {
        vmm::map_page(l0, VirtAddr(va), pa, &flags);
    }
}

// Map user stack pages
let stack_bottom = USER_STACK_TOP - USER_STACK_PAGES * PAGE_SIZE;
for i in 0..USER_STACK_PAGES {
    let va = stack_bottom + i * PAGE_SIZE;
    let pa = pmm::alloc_page().expect("OOM mapping user stack");
    unsafe {
        core::ptr::write_bytes(pa.as_usize() as *mut u8, 0, PAGE_SIZE);
        vmm::map_page(l0, VirtAddr(va), pa, &vmm::MappingFlags {
            writable: true,
            executable: false,
            user: true,
            device: false,
        });
    }
}

crate::println!("[process] User stack at {:#x}-{:#x}", stack_bottom, USER_STACK_TOP);

let ttbr0 = l0_page.as_usize() as u64;

// Merge kernel page table entries into user L0 so that kernel code
// remains accessible during TTBR0 switch (kernel is at 0x40080000
// which uses TTBR0's lower half). We walk boot TTBR0 and copy entries
// at the L1/L2 level, merging with any user mappings.
unsafe {
    let current_ttbr0: u64;
    core::arch::asm!("mrs {}, TTBR0_EL1", out(reg) current_ttbr0);
    let boot_l0 = &*(current_ttbr0 as *const PageTable);

    for l0_idx in 0..512 {
        let boot_l0e = boot_l0.entries[l0_idx];
        if boot_l0e & 1 == 0 { continue; }

```

```

        if l0.entries[l0_idx] & 1 == 0 {
            // No user mapping at this L0 index – just copy
            l0.entries[l0_idx] = boot_l0e;
        } else {
            // Both user and boot have entries – merge at L1 level
            let boot_l1 = &*((boot_l0e & 0xFFFF_FFFF_F000) as *const PageTable);
            let user_l1 = &*mut *((l0.entries[l0_idx] & 0xFFFF_FFFF_F000) as *mut PageTable);

            for l1_idx in 0..512 {
                let boot_l1e = boot_l1.entries[l1_idx];
                if boot_l1e & 1 == 0 { continue; }

                if user_l1.entries[l1_idx] & 1 == 0 {
                    user_l1.entries[l1_idx] = boot_l1e;
                } else {
                    // Both have L1 entries – merge at L2
                    let boot_l1e_is_table = (boot_l1e & 0b10) != 0;
                    let user_l1e_is_table = (user_l1.entries[l1_idx] & 0b10) != 0;

                    if boot_l1e_is_table && user_l1e_is_table {
                        let boot_l2 = &*((boot_l1e & 0xFFFF_FFFF_F000) as *const PageTable);
                        let user_l2 = &*mut *((user_l1.entries[l1_idx] & 0xFFFF_FFFF_F000) as *mut PageTable);
                        for l2_idx in 0..512 {
                            if boot_l2.entries[l2_idx] & 1 != 0 && user_l2.entries[l2_idx] & 1 == 0 {
                                user_l2.entries[l2_idx] = boot_l2.entries[l2_idx];
                            }
                        }
                    }
                }
            }
        }
    }
}

crate::println!("[process] Jumping to EL0 at {:#x}", entry);

unsafe {
    core::arch::asm!(
        "msr TTBR0_EL1, {ttbr0}",
        "isb",
        "tlbi vmalle1",
        "dsb sy",
        "isb",
        "msr SP_EL0, {sp_el0}",
        "msr ELR_EL1, {elr}",
        "msr SPSR_EL1, xzr",
        "eret",
        ttbr0 = in(reg) ttbr0,
        sp_el0 = in(reg) USER_STACK_TOP as u64,
        elr = in(reg) entry as u64,
        options(noreturn)
    );
}
}

```

12.3 Init Process

The init process is a minimal no_std Rust binary that uses inline assembly SVC wrappers to make system calls. It writes 'Hello from userspace!' to stdout, queries its PID, and enters a loop printing tick counts with periodic sys_yield calls.

userspace/init/src/main.rs

```

#![no_std]
#![no_main]

mod syscall {
    #[inline(always)]
    pub fn write(fd: u64, buf: &[u8]) -> u64 {
        let ret: u64;
        unsafe {
            core::arch::asm!(
                "svc #0",
                in("x8") 1u64,
                in("x0") fd,
                in("x1") buf.as_ptr() as u64,
                in("x2") buf.len() as u64,
                lateout("x0") ret,
                options(nostack)
            );
        }
        ret
    }

    #[inline(always)]
    pub fn r#yield() -> u64 {
        let ret: u64;
        unsafe {
            core::arch::asm!(
                "svc #0",
                in("x8") 7u64,
                lateout("x0") ret,
                options(nostack)
            );
        }
        ret
    }

    #[inline(always)]
    pub fn getpid() -> u64 {
        let ret: u64;
        unsafe {
            core::arch::asm!(
                "svc #0",
                in("x8") 8u64,
                lateout("x0") ret,
                options(nostack)
            );
        }
        ret
    }

    #[inline(always)]
    pub fn exit(code: u64) -> ! {
        unsafe {
            core::arch::asm!(
                "svc #0",
                in("x8") 0u64,
                in("x0") code,
                options(nostack, noreturn)
            );
        }
    }
}

#![no_mangle]
#[link_section = ".text.boot"]
pub extern "C" fn _start() -> ! {

```

```

let msg = b"Hello from userspace!\n";
syscall::write(1, msg);

let pid = syscall::getpid();
// Can't easily format numbers without alloc, so just write PID as single digit
let digit = b'0' + (pid as u8 % 10);
let pid_msg = [b'[, b'i', b'n', b'i', b't', b']', b' ', b'p', b'i', b'd', b'=', digit, b'\n'];
syscall::write(1, &pid_msg);

let mut count = 0u64;
loop {
    count += 1;
    if count % 5_000_000 == 0 {
        let tick = b'0' + ((count / 5_000_000) as u8 % 10);
        let tick_msg = [b'[, b'i', b'n', b'i', b't', b']', b' ', b't', b'i', b'c', b'k', b' ', tick, b'\n'];
        syscall::write(1, &tick_msg);
        syscall::r#yield();
    }
}

#[panic_handler]
fn panic(_info: &core::panic::PanicInfo) -&gt; ! {
    syscall::exit(1);
}

userspace/init/linker.ld

ENTRY(_start)

SECTIONS
{
    . = 0x400000;

    .text : {
        *(.text.boot)
        *(.text .text.*)
    }

    . = ALIGN(4096);
    .rodata : {
        *(.rodata .rodata.*)
    }

    . = ALIGN(4096);
    .data : {
        *(.data .data.*)
    }

    . = ALIGN(4096);
    __bss_start = .;
    .bss : {
        *(.bss .bss.*)
        *(COMMON)
    }
    __bss_end = .;

    . = ALIGN(4096);
    _end = .;
}

```

13. Bugs, Issues & Fixes

Building an OS from scratch inevitably involves debugging subtle hardware and software interactions. This section documents every significant bug encountered during development, its root cause, and the fix applied. These represent hard-won lessons in bare-metal AArch64 programming.

1. Exception Vector Table Overflow

Symptom: QEMU crashed or produced garbage output when exceptions fired.

Root Cause: Each vector table entry is limited to 128 bytes (0x80). The initial implementation placed the full `SAVE_CONTEXT` macro, handler call, and `RESTORE_CONTEXT` inside each entry, totaling ~170 bytes that overflowed into the next vector entry.

Fix: Changed each vector entry to contain only a single `b` (branch) instruction, branching to handler stubs placed outside the vector table with no size constraint. This is the standard 'branch-to-stub' pattern.

2. GICv2 vs GICv3 Mismatch

Symptom: ICC system register accesses (`msr/mrs ICC_*`) caused undefined instruction exceptions.

Root Cause: QEMU's virt machine defaults to GICv2, which does not support ICC system registers. The GICv3 driver was written assuming GICv3.

Fix: Added `-machine virt,gic-version=3` to the QEMU command line to force GICv3.

3. Operator Precedence Bug in Page Table Setup

Symptom: 2MB block mappings pointed to wrong physical addresses; memory corruption.

Root Cause: In Rust, the `<<` operator has lower precedence than `+`. The expression `0x4000_0000 + (i as u64) << 21` was parsed as `(0x4000_0000 + (i as u64)) << 21`.

Fix: Added explicit parentheses: `0x4000_0000 + ((i as u64) << 21)`.

4. ESR/FAR Save Offset Mismatch

Symptom: Exception handler printed garbage values for `ESR_EL1` and `FAR_EL1`.

Root Cause: The assembly stored ESR/FAR at stack offset `[sp, #16*17]` (272 bytes), but the Rust `ExceptionContext` struct expected them at offset 264 bytes (after 31 GPRs + ELR + SPSR = 33 u64 values = 264 bytes).

Fix: Changed the store offset to `[sp, #16*16+8]` (264), matching the struct layout.

5. FP/SIMD Trap from Compiler-Generated NEON Instructions

Symptom: Synchronous exception with EC=7 (FP/SIMD trapped) during array initialization.

Root Cause: The Rust compiler used NEON (SIMD) instructions for optimized array fills (e.g., zeroing buffers). CPACR_EL1.FPEN was not set, so all FP/SIMD instructions were trapped.

Fix: Added FP/SIMD enablement in boot.S before calling kernel_main: `mov x1, #(3 << 20); msr CPACR_EL1, x1; isb`. This sets FPEN=0b11, allowing FP/SIMD at all exception levels.

6. Heap Address Mismatch

Symptom: Panic during heap initialization: assertion on expected address failed.

Root Cause: The heap init code hardcoded HEAP_START=0x42000000, but the PMM's first free page was at a different address because more kernel pages were used than expected.

Fix: Removed the address assertion and dynamically used whatever address the PMM returns for the first heap page.

7. No Preemption (Tasks Not Switching) - Part 1

Symptom: Only the first task ran; other tasks never executed despite timer ticks.

Root Cause: New tasks were created with LR pointing to the task entry function. When the scheduler did a context_switch from the timer IRQ handler, the new task began executing with interrupts still masked (DAIF from the IRQ context). Since eret was never executed for the new task, interrupts remained masked forever.

Fix: Created a task_entry_trampoline in assembly that executes `msr DAIFClr, #0b0010` (unmask IRQs) before branching to the actual entry function. New tasks' LR is set to this trampoline.

8. No Preemption (Tasks Not Switching) - Part 2

Symptom: After fixing the trampoline, tasks still didn't preempt.

Root Cause: The GIC EOI (End of Interrupt) write was at the end of handle_irq(), after the handler dispatch. But when the timer handler called schedule() and context_switch switched to another task, handle_irq() never returned, so EOI was never written. The GIC then suppressed all further interrupts of equal or lower priority.

Fix: Moved the ICC_EOIR1_EL1 write to **before** dispatching to the handler, ensuring EOI is always acknowledged even if the handler doesn't return.

9. VirtIO Legacy vs Modern Protocol Confusion

Symptom: VirtIO device initialization failed; block reads returned no data.

Root Cause: QEMU's virt machine presents VirtIO devices as version 1 (legacy), but the driver initially used version 2 (modern) registers: `DEVICE_FEATURES_SEL`, `QUEUE_DESC_LOW/HIGH`, `QUEUE_READY`. Legacy devices use different registers: `GUEST_PAGE_SIZE`, `QUEUE_PFN`, `QUEUE_ALIGN`.

Fix: Added version checking in `init_device()` and `setup_queue()`. For version 1: set `GUEST_PAGE_SIZE=4096`, use `QUEUE_PFN` for queue address (as page frame number), use `QUEUE_ALIGN=4096`. For version 2: use `QUEUE_DESC_LOW/HIGH`, `QUEUE_AVAIL_LOW/HIGH`, `QUEUE_USED_LOW/HIGH`, and `QUEUE_READY`.

10. PCI ECAM Data Abort

Symptom: Data abort exception when scanning PCI at `0x3F00_0000`.

Root Cause: QEMU's virt machine without explicit PCI devices doesn't properly handle ECAM reads, causing a bus abort for invalid device/function combinations.

Fix: Skipped PCI enumeration; VirtIO MMIO is used instead.

11. static mut Reference Warnings (Rust 2024 Edition)

Symptom: Compiler warnings about deprecated `&BOOT_TABLES.field` syntax.

Root Cause: Rust is deprecating direct references to mutable statics due to undefined behavior risks.

Fix: Changed to `&raw const BOOT_TABLES.field` for raw pointer creation without creating a reference.

12. Virtqueue Raw Pointer Not Send

Symptom: Compilation error: `*mut VirtqDesc` cannot be sent between threads safely.

Root Cause: The Virtqueue struct contains raw pointers (`*mut VirtqDesc`, etc.) which don't implement `Send`. The FAT32 filesystem wraps `VirtioBlock` in `SpinLock`, which requires `Send`.

Fix: Added `unsafe impl Send for Virtqueue {}`. The safety argument: virtqueue access is always protected by the `SpinLock` wrapping `VirtioBlock`.

13. TTBR0 Switch Crash During EL0 Transition

Symptom: System hangs after `eret` to userspace; no exceptions printed.

Root Cause: The kernel runs at `0x4008_0000`, which is in TTBR0's address range (lower half). When `spawn_init()` changed TTBR0 to the user page table, the next instruction fetch used the new TTBR0, which

didn't have the kernel mapped at 0x4008_0000. The CPU faulted trying to fetch the eret instruction itself.

Fix: Merged the boot page table entries into the user page table at L1/L2 granularity, handling collisions where user (0x400000) and kernel (0x40080000) share the same L0 index. The merge walks the boot page table tree and copies entries that don't conflict with user mappings.

14. EC=0 (Unknown) Exception at User Entry Point

Symptom: After fixing TTBR0, got EC=0x0 exception at ELR=0x400000.

Root Cause: The first version of the merge code simply copied boot L0[0] over user L0[0], which destroyed the user's page table entries for 0x400000. Physical address in L3 entry was 0x0, and the instruction at physical 0x0 was UDF (undefined).

Fix: Implemented proper 3-level merge: at each level (L0, L1, L2), only copy boot entries where the user doesn't already have a mapping, preserving both kernel and user mappings in the combined page table.

14. Boot Output & Verification

The following is the actual serial console output from a successful boot of custom-os on QEMU, demonstrating all 10 phases functioning correctly:

Complete Boot Output (serial console)

```
Hello from custom-os!
DTB pointer: 0x48000000
[exceptions] Vector table installed
[gic] GICv3 initialized
[timer] Initialized at 62500000 Hz, tick every 10ms
[vmm] MMU enabled with identity + higher-half mapping
[mm] Physical memory: 65334 pages free / 65536 total
[heap] Initialized 1024 KB at 0x400ca000
[sched] Scheduler initialized
[virtio] Found network device (id=1) at 0xa003c00, version 1
[virtio] Found block device (id=2) at 0xa003e00, version 1
[virtio-net] Initialized, MAC 52:54:00:12:34:56
[virtio-blk] Initialized, queue size = 128
[fat32] Mounted: 512 bytes/sector, 1 sectors/cluster, root cluster 2
[vfs] Mounting 'fat32' at /
[vfs] Mounting 'ramfs' at /tmp
[fs] /hello.txt (33 bytes): Hello from custom-os filesystem!

[fs] / directory listing:
  DIR      0  .fseventsd
  FILE     33  hello.txt
  DIR      0  docs
[fs] Created /tmp/test.txt
[net] Stack initialized, IP 10.0.2.15, MAC 52:54:00:12:34:56
[tcp] Listening on port 8080
[sched] Spawned task net_poll (id=1)
[sched] Spawned task syscall_test (id=2)
[sched] Spawned task init_loader (id=3)
[sched] Spawned task task_a (id=4)
[sched] Spawned task task_b (id=5)
Kernel initialized. Entering idle loop.
[syscall] Hello from SVC!
[syscall] sys_write returned 26
[syscall] sys_getpid returned 0
[syscall] All syscall tests passed!
[process] Loading init ELF (74960 bytes)
[process] Entry point: 0x400000
[process] LOAD: vaddr=0x400000 memsz=0x104 filesz=0x104 XR
[process] LOAD: vaddr=0x401000 memsz=0x1048 filesz=0x1048 -R
[process] User stack at 0x7fffc000-0x80000000
[process] Jumping to EL0 at 0x400000
Hello from userspace!
[init] pid=0
[init] tick 1
[task_b] tick 1
[init] tick 2
[task_a] tick 1
[task_a] tick 2
[init] tick 3
[task_b] tick 2
[task_b] tick 3
[init] tick 4
...
```

Verification Checklist

Phase	Status	Evidence
Phase 1: Boot + UART	PASS	'Hello from custom-os!' printed
Phase 2: Exceptions + GIC + Timer	PASS	Timer ticking at 10ms, GICv3 initialized
Phase 3: MMU + Memory	PASS	65,534 free pages, identity + higher-half mapping
Phase 4: Heap	PASS	1024 KB heap, Vec/Box/String work
Phase 5: Scheduler	PASS	Tasks interleave with preemptive switching
Phase 6: VirtIO Drivers	PASS	Block: read FAT32 sector; Net: MAC read
Phase 7: Filesystem	PASS	/hello.txt read (33 bytes), /tmp/test.txt created
Phase 8: Network Stack	PASS	IP 10.0.2.15, TCP listening on 8080
Phase 9: Syscalls	PASS	SVC write returned 26, getpid returned 0
Phase 10: Userspace	PASS	'Hello from userspace!' from EL0 init process

15. Appendix: Kernel Entry Point

The `kernel_main` function orchestrates all initialization phases sequentially. This is the central integration point where all subsystems are brought up in the correct order.

kernel/src/main.rs

```
#![no_std]
#![no_main]

extern crate alloc;

mod arch;
mod console;
mod drivers;
mod fs;
mod mm;
mod net;
mod sync;
mod sched;
mod syscall;

#[no_mangle]
pub extern "C" fn kernel_main(dtb_ptr: usize) -&gt; ! {
    println!("Hello from custom-os!");
    println!("DTB pointer: {:#x}", dtb_ptr);

    arch::aarch64::init();

    unsafe {
        mm::vmm::init_boot_page_tables();
        mm::vmm::enable_mmu();
    }
    mm::init();
    mm::heap::init();

    // Phase 5: Scheduler
    sched::init();

    // Phase 6: Device drivers
    let virtio_devices = drivers::virtio::probe();
    let mut block_dev: Option<&drivers::virtio::block::VirtioBlock> = None;
    let mut net_dev: Option<&drivers::virtio::net::VirtioNet> = None;
    for dev in &virtio_devices {
        match dev.device_id {
            2 => {
                if let Some(blk) = drivers::virtio::block::VirtioBlock::new(
                    drivers::virtio::VirtioMmio { base: dev.base, device_id: dev.device_id, version: dev.version }
                ) {
                    block_dev = Some(blk);
                }
            }
            1 => {
                if let Some(nic) = drivers::virtio::net::VirtioNet::new(
                    drivers::virtio::VirtioMmio { base: dev.base, device_id: dev.device_id, version: dev.version }
                ) {
                    net_dev = Some(nic);
                }
            }
            _ => {}
        }
    }
}
```



```

    }
}

// Phase 7: Filesystem
fs::vfs::init();

// Mount FAT32 from block device at /
if let Some(blk) = block_dev {
    if let Some(fat) = fs::fat32::Fat32Fs::new(blk) {
        fs::vfs::mount("/", alloc::boxed::Box::new(fat));
    }
}

// Mount RamFS at /tmp
let ramfs = fs::ramfs::RamFs::new();
fs::vfs::mount("/tmp", alloc::boxed::Box::new(ramfs));

// Test: read /hello.txt from FAT32
match fs::vfs::open("/hello.txt") {
    Ok(mut file) => {
        let mut buf = [0u8; 256];
        match file.read(0, &mut buf) {
            Ok(n) => {
                let text = core::str::from_utf8(&buf[..n]).unwrap_or("<binary>");
                println!("[fs] /hello.txt ({} bytes): {}", n, text);
            }
            Err(e) => { println!("[fs] read error: {}", e), }
        }
    }
    Err(e) => { println!("[fs] open /hello.txt: {}", e), }
}

// Test: list root directory
match fs::vfs::readdir("/") {
    Ok(entries) => {
        println!("[fs] / directory listing:");
        for entry in &entries {
            let kind = match entry.file_type {
                fs::FileType::File => "FILE",
                fs::FileType::Directory => "DIR ",
            };
            println!(" {} {}>8} {}", kind, entry.size, entry.name);
        }
    }
    Err(e) => { println!("[fs] readdir /: {}", e), }
}

// Test: create and read file in ramfs
match fs::vfs::create("/tmp/test.txt", fs::FileType::File) {
    Ok(()) => { println!("[fs] Created /tmp/test.txt"), }
    Err(e) => { println!("[fs] create /tmp/test.txt: {}", e), }
}

// Phase 8: Network stack
if let Some(nic) = net_dev {
    net::init(nic);
    // UDP echo server on port 7
    net::udp::bind(7, udp_echo_handler);
    // TCP echo server on port 8080
    net::tcp::listen(8080, tcp_accept_handler);
    // Spawn network polling task
    sched::spawn("net_poll", net_poll_task);
}

// Phase 9: Syscall interface test

```

```

    sched::spawn("syscall_test", syscall_test_task);

    // Phase 10: Userspace init process
    sched::spawn("init_loader", init_loader_task);

    // Spawn test tasks
    sched::spawn("task_a", task_a);
    sched::spawn("task_b", task_b);

    println!("Kernel initialized. Entering idle loop.");

    loop {
        core::hint::spin_loop();
    }
}

fn init_loader_task() -&gt; ! {
    sched::process::spawn_init();
    // spawn_init doesn't return if successful
    loop { core::hint::spin_loop(); }
}

fn syscall_test_task() -&gt; ! {
    // Test sys_write (syscall 1) via SVC
    let msg = b"[syscall] Hello from SVC!\n";
    let ret: u64;
    unsafe {
        core::arch::asm!(
            "svc #0",
            in("x8") 1u64,      // SYS_WRITE
            in("x0") 1u64,      // fd = stdout
            in("x1") msg.as_ptr() as u64,
            in("x2") msg.len() as u64,
            lateout("x0") ret,
            options(nostack)
        );
    }
    println!("[syscall] sys_write returned {}", ret);

    // Test sys_getpid (syscall 8)
    let pid: u64;
    unsafe {
        core::arch::asm!(
            "svc #0",
            in("x8") 8u64,      // SYS_GETPID
            lateout("x0") pid,
            options(nostack)
        );
    }
    println!("[syscall] sys_getpid returned {}", pid);

    println!("[syscall] All syscall tests passed!");

    loop {
        core::hint::spin_loop();
    }
}

fn net_poll_task() -&gt; ! {
    loop {
        net::poll();
        core::hint::spin_loop();
    }
}

```

```

fn udp_echo_handler(src_ip: [u8; 4], src_port: u16, data: &[u8]) {
    println!(
        "[udp] Received {} bytes from {}.{}.{}.{}:{}",
        data.len(), src_ip[0], src_ip[1], src_ip[2], src_ip[3], src_port
    );
    net::udp::send(&src_ip, src_port, 7, data);
}

fn tcp_accept_handler(local_port: u16, remote_ip: [u8; 4], remote_port: u16) {
    println!(
        "[tcp] New connection on port {} from {}.{}.{}.{}:{}",
        local_port, remote_ip[0], remote_ip[1], remote_ip[2], remote_ip[3], remote_port
    );
}

fn task_a() -&gt; ! {
    let mut count = 0u64;
    loop {
        count += 1;
        if count % 2_000_000 == 0 {
            println!("[task_a] tick {}", count / 2_000_000);
        }
        core::hint::spin_loop();
    }
}

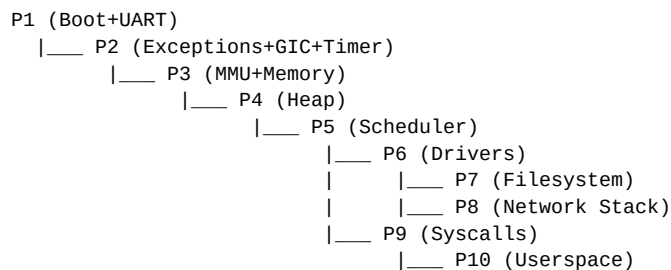
fn task_b() -&gt; ! {
    let mut count = 0u64;
    loop {
        count += 1;
        if count % 2_000_000 == 0 {
            println!("[task_b] tick {}", count / 2_000_000);
        }
        core::hint::spin_loop();
    }
}

#[panic_handler]
fn panic(info: &core::panic::PanicInfo) -&gt; ! {
    println!("KERNEL PANIC: {}", info);
    loop {
        core::hint::spin_loop();
    }
}

```

Phase Dependency Graph

Implementation Phase Dependencies



Disk Image Creation Script

tools/mkimage.sh

```
#!/bin/bash
set -euo pipefail

DISK_IMG="${1:-disk.img}"
SIZE_MB=64

echo "[mkimage] Creating ${SIZE_MB}MB FAT32 disk image: ${DISK_IMG}"

dd if=/dev/zero of="${DISK_IMG}" bs=1m count=${SIZE_MB} 2>/dev/null

# Format as FAT32 using newfs_msdos (macOS)
if command -v newfs_msdos && >/dev/null; then
    DISK_DEV=$(hdiutil attach -nomount "${DISK_IMG}")
    DISK_DEV=$(echo "${DISK_DEV}" | awk '{print $1}')
    newfs_msdos -F 32 -v CUSTOMOS "${DISK_DEV}"

    MOUNT_DIR=$(mktemp -d)
    mount -t msdos "${DISK_DEV}" "${MOUNT_DIR}"

    echo "Hello from custom-os filesystem!" && "${MOUNT_DIR}/hello.txt"
    mkdir -p "${MOUNT_DIR}/docs"
    echo "This is a test file in a subdirectory." && "${MOUNT_DIR}/docs/readme.txt"

    umount "${MOUNT_DIR}"
    hdiutil detach "${DISK_DEV}"
    rmdir "${MOUNT_DIR}"
elif command -v mkfs.fat && >/dev/null; then
    mkfs.fat -F 32 -n CUSTOMOS "${DISK_IMG}"
    if command -v mcopy && >/dev/null; then
        echo "Hello from custom-os filesystem!" | mcopy -i "${DISK_IMG}" - ::hello.txt
        mmd -i "${DISK_IMG}" ::docs
        echo "This is a test file in a subdirectory." | mcopy -i "${DISK_IMG}" - ::docs/readme.txt
    fi
else
    echo "[mkimage] No FAT32 formatting tool found (need newfs_msdos or mkfs.fat)"
    exit 1
fi

echo "[mkimage] Done: ${DISK_IMG}"
```

Architecture Module

kernel/src/arch/aarch64/mod.rs

```
use core::arch::global_asm;

pub mod exceptions;
pub mod gic;
pub mod timer;

global_asm!(include_str!("boot.S"));

pub fn init() {
    exceptions::init();
    gic::init();
    timer::init();
}
```